



AI EMPIRE ACADEMY – GIÁO TRÌNH CHUYÊN ĐỀ

BỘ NHỚ HỘI THOẠI VỚI LANGGRAPH

Từ message và thread đến context dài, Store và
FastAPI

Giáo trình thực hành từ nền tảng đến chatbot nhiều cuộc hội thoại

Biên soạn: Mr. Tran Minh Nhat

Đơn vị: AI Empire Academy

Liên hệ: 0976059814

Website: aiempireacademy.com.vn

Phiên bản học thuật 2026

Mục lục

Lời mở đầu	vii
Cách học và kế hoạch 10 buổi	ix
1 MODEL KHÔNG TỰ NHỚ	1
1.1 Nguyên lý Stateless của LLM và Giao thức HTTP	1
1.2 Ba lớp thường bị gọi chung là “lịch sử”	1
1.3 Xây dựng bộ nhớ thủ công bằng Python thuần	2
1.3.1 Phân tích chi tiết mã nguồn	3
1.4 Vì sao danh sách thuần chưa đủ cho backend?	4
2 STATE, NODE, EDGE VÀ GRAPH	5
2.1 State là hợp đồng dữ liệu	5
2.2 Node là hàm Python	6
2.2.1 Lưu ý quan trọng khi viết Node	6
2.3 Edge định nghĩa thứ tự thực thi	6
2.3.1 Giải thích từng dòng code trong <code>graph_basics.py</code>	7
2.4 Compile không đồng nghĩa với Persistence	7
2.5 Đọc đồ thị bằng năm câu hỏi vàng	8
3 SHORT-TERM MEMORY THEO THREAD	9
3.1 Tích hợp InMemorySaver vào đồ thị	9
3.1.1 Phân tích chi tiết mã nguồn <code>short_term_memory.py</code>	10
3.2 Dòng thời gian thực thi của hai luồng hội thoại độc lập	11
3.3 Nguyên tắc gửi tin nhắn mới từ phía Client	11
3.4 Sự khác biệt giữa Thread ID và Quyền truy cập	12
3.5 Đọc trạng thái đồ thị phục vụ Debug và Hiển thị lịch sử	12
3.5.1 Lưu ý khi sử dụng <code>get_state</code>	12
4 QUẢN LÝ CONTEXT DÀI	14
4.1 Vì sao ngữ cảnh quá dài không hề miễn phí?	14
4.2 Cắt bớt tin nhắn bằng kỹ thuật tạo View ngữ cảnh	15
4.2.1 Phân tích chi tiết hàm <code>context_for_model</code>	15
4.3 Không cắt tin nhắn một cách mù quáng	16
4.4 Mẫu phân bổ ngân sách ngữ cảnh hợp lý	16

5	RUNNING SUMMARY (TÓM TẮT TĂNG DẦN)	18
5.1	Mở rộng State Schema để chứa Summary	18
5.1.1	Bản chất của trường dữ liệu <code>summary</code>	18
5.2	Chu trình hoạt động của Node tóm tắt lũy tiến	18
5.3	Tiêu chí của một bản tóm tắt chất lượng cao	19
5.3.1	Thông tin cần lược bỏ	19
5.4	Mô phỏng cơ chế Compact tin nhắn trong mã nguồn	20
5.4.1	Phân tích chi tiết mã nguồn	20
5.5	Lựa chọn thời điểm Trigger chạy tóm tắt	21
6	LONG-TERM MEMORY THEO USER	22
6.1	Thiết kế cấu trúc Namespace và Key	22
6.1.1	Phân tích chi tiết mã nguồn <code>long_term_memory.py</code>	23
6.2	Phân loại bộ nhớ dài hạn đáng lưu trữ	23
6.3	Giải pháp kết hợp Thread và User trong một Dịch vụ hội thoại	24
6.3.1	Phân tích chi tiết luồng xử lý trong <code>ChatService</code>	25
7	PERSISTENCE VÀ RANH GIỚI PRODUCTION	27
7.1	Chọn lựa cơ sở dữ liệu lưu trữ theo từng giai đoạn	27
7.2	Xử lý bất đồng bộ khi tích hợp Cơ sở dữ liệu	27
7.3	Ranh giới Phân quyền sở hữu dữ liệu (Ownership)	28
7.4	Quy định về thời hạn lưu trữ (Retention) và Xóa dữ liệu	29
8	TÍCH HỢP VÀO LLM GATEWAY	30
8.1	Hợp đồng API đề xuất cho cổng dịch vụ (LLM Gateway)	30
8.1.1	Luồng dữ liệu giữa Client và Server	30
8.2	Xây dựng API hoàn chỉnh bằng FastAPI	31
8.2.1	Phân tích chi tiết mã nguồn <code>fastapi_app.py</code>	32
8.3	Adapter tích hợp mô hình thực tế (DeepSeek và Gemini)	32
8.3.1	Phân tích chi tiết mã nguồn <code>real_model_adapter.py</code>	33
8.4	Lộ trình di cư mã nguồn (Migration Slices)	34
9	KIỂM THỬ, DEBUG VÀ QUAN SÁT	35
9.1	Bốn quy tắc bất biến cốt lõi về bộ nhớ	35
9.1.1	Phân tích chi tiết mã nguồn <code>test_memory.py</code>	36
9.2	Kiểm thử tầng API, bảo vệ ranh giới bảo mật	37
9.2.1	Phân tích chi tiết mã nguồn <code>test_fastapi_app.py</code>	37
9.3	Failure Lab: Các lỗi kinh điển và cách xử lý	38
9.4	Giám sát Vận hành (Telemetry & Metrics)	38
10	DỰ ÁN CUỐI KHÓA: CHATBOT NHIỀU CUỘC HỘI THOẠI	40
10.1	Mô tả đề tài và Mục tiêu học tập	40
10.2	Yêu cầu chức năng chi tiết	40
10.3	Yêu cầu phi chức năng và Vận hành	41
10.4	Kiến trúc tham chiếu hệ thống Capstone	41

10.5 Kế hoạch thực hiện theo mốc kiểm chứng	41
10.6 Kịch bản Demo 5 phút bắt buộc	42
10.7 Định nghĩa hoàn thành (Definition of Done)	43
Kết luận	44
A Cài đặt và lệnh chạy	45
A.1 Cài môi trường	45
A.2 Provider thật	45
A.3 Build PDF	45
A.4 Fallback	46
B Bài tập tổng hợp và rubric	47
B.1 Bài tập cấp 1: mô hình tinh thần	47
B.2 Bài tập cấp 2: LangGraph memory	47
B.3 Bài tập cấp 3: production	47
B.4 Rubric dự án cuối khóa	48
B.5 Thang phản hồi bốn mức	48
C Thuật ngữ nhanh	49
Tài liệu tham khảo	50

Danh sách hình vẽ

1.1	Hai request độc lập khi ứng dụng không gửi lại lịch sử.	1
2.1	Graph hội thoại nhỏ nhất chưa có persistence.	6
3.1	Cùng graph và checkpoint, state được phân vùng bởi thread.	11
4.1	Trim tạo view cho model nhưng có thể giữ storage đầy đủ.	16
5.1	Running summary chỉ chạy khi history vượt ngưỡng.	19
6.1	Nhiều thread của cùng user truy cập một namespace memory.	26
8.1	Ranh giới thành phần trong LLM Gateway có memory.	33
10.1	Kiến trúc tham chiếu cho dự án cuối khóa.	41

Danh sách bảng

1	Kế hoạch học theo từng buổi 90 phút	ix
1.1	Ba lớp history trong một ứng dụng chat	2
4.1	Ví dụ phân bổ ngữ cảnh ngữ âm (Token Budget)	17
6.1	So sánh giữa Checkpointer và Store	22
7.1	Lộ trình backend cho bộ nhớ tác tử	27
9.1	Các chỉ số giám sát vận hành bộ nhớ tác tử	39
10.1	Các mốc dự án có thể kiểm chứng	42
B.1	Rubric 100 điểm cho chatbot có memory	48
C.1	Thuật ngữ cốt lõi	49

Lời mở đầu

Một giao diện có bong bóng tin nhắn chưa chắc đã là một chatbot có trí nhớ. Model ngôn ngữ chỉ thấy dữ liệu nằm trong request hiện tại; nếu ứng dụng không gửi lại thông tin cũ, câu hỏi “Tôi tên gì?” không có cách nào liên hệ với câu “Tôi tên Nhật” ở request trước. Từ quan sát nhỏ này xuất hiện cả một lớp bài toán hệ thống: lưu message ở đâu, phân tách nhiều cuộc trò chuyện thế nào, đưa bao nhiêu lịch sử vào context, tóm tắt phần cũ ra sao, và thông tin nào được phép sống lâu hơn một thread.

Giáo trình này xây mô hình tinh thần trước rồi mới thêm framework. Học viên bắt đầu bằng một danh sách Python thuần, chuyển nó thành state của LangGraph, thêm checkpoint, quan sát hai `thread_id` độc lập, rồi mới học trimming, running summary và long-term Store. Phần cuối ghép các mảnh ấy vào FastAPI và thiết kế API cho một LLM Gateway nhiều provider.

Định nghĩa: Phạm vi

Cuốn sách tập trung vào bộ nhớ hội thoại và context engineering. Qdrant, RAG và agent gọi tool chỉ được nhắc ở ranh giới cần phân biệt; chúng không phải điều kiện để hoàn thành các lab.

Thực giác: Nguyên tắc xuyên suốt

Model không sở hữu trí nhớ. Ứng dụng sở hữu dữ liệu; LangGraph giúp tổ chức state, checkpoint và luồng cập nhật; model chỉ nhận một lát context do ứng dụng lựa chọn ở thời điểm gọi.

Mọi ví dụ cốt lõi dùng một “model quyết định” bằng quy tắc. Nhờ vậy học viên kiểm tra được memory mà không phụ thuộc mạng, API key hay độ ngẫu nhiên của LLM. Khi cơ chế đã đúng, ta thay model giả bằng DeepSeek hoặc Gemini mà không đổi invariant của hội thoại.

Đối tượng và điều kiện đầu vào

Tài liệu dành cho người đã biết Python cơ bản, dictionary, list, function và cách chạy một ứng dụng FastAPI; không giả định đã dùng LangChain hay LangGraph. Python 3.11 trở lên và công cụ uv là cấu hình đề xuất. LangGraph hiện yêu cầu Python 3.10 trở lên và được cài độc lập với các gói provider (LangChain 2026b).

Kết quả đầu ra quan sát được

Sau khi hoàn thành, học viên có thể:

1. giải thích vì sao UI history khác model context và persistent history;
2. dựng graph `START -> call_model -> END`;
3. dùng checkpointer và `thread_id` để tách cuộc trò chuyện;
4. chọn trim hoặc summary dựa trên chi phí và yêu cầu giữ thông tin;
5. dùng `user_id` và Store cho memory xuyên thread;
6. thiết kế API chống trộn dữ liệu giữa người dùng;
7. viết test chứng minh các invariant trên.

Cách học và kế hoạch 10 buổi

Điểm tự kiểm tra: Nhịp học đề xuất

Mỗi chương theo vòng lặp: dự đoán trước khi chạy; chạy ví dụ nhỏ nhất; vẽ lại state; cố ý dùng sai `thread_id`; chạy test; cuối cùng áp dụng vào LLM Gateway. Học viên nên chủ động ít nhất một nửa thời lượng thay vì chỉ xem giảng viên gõ code.

Bảng 1: Kế hoạch học theo từng buổi 90 phút

Buổi	Trọng tâm	Thực hành	Minh chứng hoàn thành
1	Model context và ba lớp history	Python thuần	Model giả trả đúng tên khi có history.
2	State, node, edge, graph	Graph nhỏ nhất	Vẽ và chạy được graph một node.
3	Checkpoint, thread, checkpoint	Hai thread	Nhật và An không bị trộn tên.
4	Context window và token budget	Trim	Chứng minh storage đầy đủ nhưng model input ngắn.
5	Running summary	Compact history	Summary giữ quyết định, message gần vẫn nguyên văn.
6	Store và user memory	Hai user, nhiều thread	Memory đi qua thread nhưng không đi qua user.
7	SQLite/PostgreSQL	Thiết kế persistence	Chọn backend và mô tả vòng đời dữ liệu.
8	FastAPI/Streamlit	Conversation API	Tạo, chat, đọc và xóa một thread.
9	Test, log, privacy	Failure lab	Test ownership và test restart.
10	Dự án cuối khóa	LLM Gateway	Demo end-to-end và báo cáo trade-off.

Cấu trúc một buổi 90 phút

- **0–8 phút:** kiểm tra mục tiêu và dự đoán output.
- **8–20 phút:** mô hình tinh thần bằng một ví dụ cụ thể.
- **20–35 phút:** giảng viên demo và nói rõ state thay đổi ở đâu.
- **35–55 phút:** thực hành có hướng dẫn, gồm một lỗi cố ý.

- **55–75 phút:** học viên tự áp dụng vào biến thể gần dự án thật.
- **75–85 phút:** chạy test, phản hồi theo rubric, sửa một lần.
- **85–90 phút:** chốt artifact và bài tập về nhà.

Cách đọc code

Mỗi listing quan trọng trong sách lấy từ thư mục `code/`. Trước khi chạy, hãy trả lời bốn câu: input mới là gì; state cũ nằm ở đâu; hàm nào thay state; output nào được lưu lại. Nếu chưa trả lời được, đừng vội đổi sang model thật.

Cảnh báo: Fallback khi không có API key

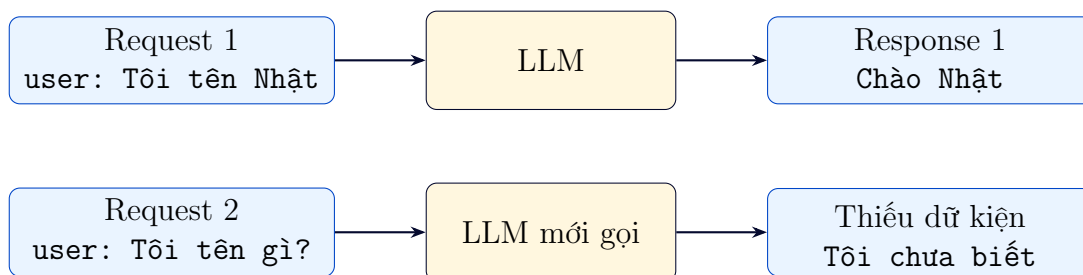
Không bỏ qua lab. Model quyết định trong tài liệu cố ý không cần API. Memory là thuộc tính của ứng dụng; ta có thể kiểm thử chính xác mà không cần một LLM sinh văn bản.

1. MODEL KHÔNG TỰ NHỚ

1.1. Nguyên lý Stateless của LLM và Giao thức HTTP

Mọi mô hình ngôn ngữ lớn (LLM), dù là DeepSeek, Gemini hay GPT, đều hoạt động theo mô hình không trạng thái (stateless) ở tầng API. Khi bạn gửi một yêu cầu (request) đến máy chủ của nhà cung cấp LLM, yêu cầu đó được đóng gói và xử lý độc lập hoàn toàn với các yêu cầu trước hoặc sau nó. Máy chủ LLM không duy trì bất kỳ bộ nhớ đệm hay phiên làm việc nào lưu lại vết các câu hỏi trước đó của bạn trên cùng một kênh kết nối mạng.

Hãy coi mỗi lần gọi model là gửi một phong bì thư kín. Mô hình chỉ đọc các tờ giấy nằm trong chính phong bì đó để tạo ra câu trả lời và ngay lập tức quên đi nội dung sau khi gửi thư phản hồi (response) lại cho bạn. Do đó, request đầu tiên chứa thông tin “Tôi tên Nhật” sẽ không tự động xuất hiện trong request thứ hai chứa câu hỏi “Tôi tên gì?”. Nếu ứng dụng client không tự động đính kèm lịch sử trò chuyện vào phong bì mới, mô hình sẽ hoàn toàn không có dữ liệu để trả lời và sẽ báo lỗi hoặc trả lời rằng nó chưa biết thông tin đó.



Hình 1.1: Hai request độc lập khi ứng dụng không gửi lại lịch sử.

Nguồn: Tác giả tự dựng.

1.2. Ba lớp thường bị gọi chung là “lịch sử”

Trong một ứng dụng chatbot hoàn chỉnh, từ “lịch sử” (history) thường được sử dụng rất mơ hồ và dễ dẫn đến nhầm lẫn trong thiết kế hệ thống. Để xây dựng hệ thống bộ nhớ chuẩn xác, chúng ta cần phân biệt rõ ba lớp lưu trữ lịch sử sau đây:

Bảng 1.1: Ba lớp history trong một ứng dụng chat

Lớp	Mục đích	Ví dụ
UI history	Vẽ lại bong bóng chat trên giao diện người dùng	Biến cục bộ trong frontend như <code>st.session_state.messages</code> trong Streamlit hoặc state React.
Persistent history	Lưu trữ lâu dài và khôi phục sau khi reload trang hoặc restart server	Lưu trữ vật lý trong cơ sở dữ liệu như SQLite, PostgreSQL, MongoDB hoặc các checkpoint của LangGraph.
Model context	Lát cắt dữ liệu thực tế được đưa vào prompt gửi đến LLM API	Tổ hợp bao gồm: System prompt + Lược sử đã rút gọn (summary) + Các message hội thoại gần nhất.

Nhà phát triển rất dễ phạm phải hai sai lầm đối lập:

1. **Lỗi hiển thị ảo:** Thấy giao diện UI hiển thị đầy đủ bong bóng hội thoại cũ nên lầm tưởng mô hình LLM đã “nhớ” mọi chuyện, trong khi thực chất mã nguồn backend chỉ gửi duy nhất tin nhắn mới nhất đến LLM.
2. **Lỗi quá tải context:** Lưu trữ được bao nhiêu tin nhắn trong database thì gửi toàn bộ bấy nhiêu tin nhắn vào LLM ở mỗi lượt gọi. Lỗi này khiến chi phí token đầu vào tăng phi mã và làm tăng độ trễ (latency) của mô hình một cách không cần thiết.

1.3. Xây dựng bộ nhớ thủ công bằng Python thuần

Để hiểu cách các framework như LangGraph tự động hóa quy trình quản lý lịch sử, trước tiên chúng ta hãy tự xây dựng một giải pháp quản lý bộ nhớ thủ công bằng Python thuần thông qua mã nguồn `manual_history.py`.

Mã nguồn 1.1: Giữ lịch sử bằng Python thuần

```

"""Minimal example: the application owns its conversation history."""

from dataclasses import dataclass, field
import sys

Message = dict[str, str]

def fake_model(messages: list[Message]) -> str:
    """A deterministic model keeps the example independent from API keys."""
    latest = messages[-1]["content"].lower()
    if "t\u00f4i t\u00e2n g\u00ec" in latest:
        for message in reversed(messages[:-1]):
            content = message["content"]
            marker = "T\u00f4i t\u00e2n "
            if message["role"] == "user" and marker in content:
                return f"B\u1ea1n t\u00e2n {content.split(marker, 1)[1].rstrip('.')}.replace(" .", ".")
            return "T\u00f4i ch\u01b0a bi\u1ebft t\u00e2n b\u1ea1n."
    return "T\u00f4i \u0111\u00e3 nh\u1eadn \u0111\u01b0\u1ee3c tin nh\u1ea5n."

```

```
@dataclass
class ManualConversation:
    messages: list[Message] = field(default_factory=list)

    def ask(self, text: str) -> str:
        self.messages.append({"role": "user", "content": text})
        answer = fake_model(self.messages)
        self.messages.append({"role": "assistant", "content": answer})
        return answer

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    chat = ManualConversation()
    print(chat.ask("T\u00f4i t\u00ean Nh\u01eadt."))
    print(chat.ask("T\u00f4i t\u00ean g\u00ec?"))
```

1.3.1. Phân tích chi tiết mã nguồn

- **Định nghĩa kiểu dữ liệu Message:** Dòng `Message = dict[str, str]` định nghĩa cấu trúc của một tin nhắn dưới dạng từ điển (dictionary) có hai khóa chính: `"role"` (đại diện cho vai trò gửi tin nhắn như `"user"`, `"assistant"`, hoặc `"system"`) và `"content"` (nội dung văn bản của tin nhắn).
- **Hàm giả lập LLM `fake_model`:** Hàm này nhận vào danh sách lịch sử tin nhắn `messages`. Nhằm tránh phụ thuộc vào API key, hàm sử dụng các quy tắc logic đơn giản:
 - Nếu tin nhắn cuối cùng chứa câu hỏi tên (“tôi tên gì”), hàm sẽ duyệt ngược danh sách các tin nhắn cũ từ dưới lên để tìm tin nhắn của người dùng chứa cụm từ “Tôi tên ”. Nếu tìm thấy, nó sẽ trích xuất tên và trả về.
 - Nếu không tìm thấy thông tin tên trong lịch sử, nó trả về thông điệp mặc định “Tôi chưa biết tên bạn.”.
- **Class quản lý hội thoại `ManualConversation`:** Class này sử dụng decorator `@dataclass` để tự động tạo phương thức khởi tạo `__init__`. Trường `messages: list[Message]` được gán giá trị mặc định là một danh sách rỗng thông qua `field(default_factory=list)`.
- **Hàm xử lý hội thoại `ask`:** Hàm này thực thi theo đúng quy trình 3 bước nền tảng của một chatbot:
 1. Đưa tin nhắn mới của người dùng vào lịch sử lưu trữ: `self.messages.append(...)`.
 2. Gọi mô hình (ở đây là `fake_model`) bằng cách truyền toàn bộ danh sách lịch sử `self.messages`.
 3. Đưa câu trả lời của mô hình vào lịch sử lưu trữ và trả lại kết quả cho người dùng.

Ví dụ: Dự đoán trước khi chạy

Nếu tạo hai đối tượng độc lập `chat1 = ManualConversation()` và `chat2 = ManualConversation()`, sau đó gọi `chat1.ask("Tôi tên Nhật.")` và `chat2.ask("Tôi tên gì?")`, kết quả của câu gọi thứ hai chắc chắn sẽ là "Tôi chưa biết tên bạn.". Điều này chứng minh mỗi đối tượng lớp sở hữu một vùng nhớ độc lập hoàn toàn.

1.4. Vì sao danh sách thuần chưa đủ cho backend?

Mặc dù giải pháp lưu trữ lịch sử bằng danh sách trong bộ nhớ RAM hoạt động tốt đối với một script chạy đơn lẻ trên local, nhưng khi đưa lên môi trường web backend (ví dụ như FastAPI) chạy thực tế, chúng ta sẽ gặp tiếp đối mặt với những bài toán phức tạp sau:

- **Định danh luồng (Thread Identification):** Làm cách nào để phân biệt tin nhắn gửi lên thuộc về cuộc hội thoại nào khi có hàng trăm request đồng thời từ nhiều người dùng khác nhau gửi về backend?
- **Bảo mật và phân quyền (Authorization):** Làm sao đảm bảo người dùng A không thể đọc được lịch sử trò chuyện của người dùng B thông qua việc đoán hoặc quét ID?
- **Mất mát dữ liệu khi khởi động lại (Volatility):** Dữ liệu lưu hoàn toàn trên RAM sẽ biến mất ngay khi tiến trình ứng dụng khởi động lại hoặc khi ứng dụng tự động scale-out thêm các worker mới.
- **Giới hạn kích thước ngữ cảnh (Context Overflow):** Làm cách nào để tránh việc gửi một danh sách lịch sử quá dài vượt ngưỡng xử lý tối đa của mô hình?

Định nghĩa: Hai định danh cốt lõi

Để giải quyết các vấn đề trên, hệ thống cần quản lý chặt chẽ hai định danh độc lập:

1. **thread_id:** Định danh duy nhất cho một phiên trò chuyện cụ thể.
2. **user_id:** Định danh duy nhất cho một người dùng cụ thể. Một người dùng có thể sở hữu nhiều **thread_id** khác nhau, nhưng một **thread_id** chỉ thuộc quyền sở hữu của một **user_id** duy nhất.

Điểm tự kiểm tra: Lab 15 phút

Chạy thử tệp `manual_history.py`. Sau đó, viết thêm một đoạn mã khởi tạo song song hai đối tượng hội thoại `chat_a` và `chat_b`. Thực hiện tuần tự việc khai báo tên ở từng cuộc hội thoại và hỏi lại để đảm bảo dữ liệu không bị trộn lẫn giữa các đối tượng.

2. STATE, NODE, EDGE VÀ GRAPH

LangGraph là hạ tầng mức thấp cho phép xây dựng các quy trình làm việc (workflow) có trạng thái (stateful) và chạy lâu dài; bản thân nó không tự đưa ra quyết định về prompt, cấu trúc tác tử (agent), hay kiến trúc ứng dụng cụ thể nào cả (LangChain 2026c). Để học LangGraph hiệu quả nhất, chúng ta cần đi từ những khối gạch cơ bản nhất: dựng một đồ thị (graph) cực tiểu, quan sát dữ liệu đầu vào/đầu ra, và sau đó mới tích hợp cơ chế bộ nhớ.

2.1. State là hợp đồng dữ liệu

State (trạng thái) đại diện cho một ảnh chụp dữ liệu (data snapshot) mà các node trong đồ thị cùng đọc và cập nhật. Trong một ứng dụng chatbot, cấu trúc state tối thiểu cần phải chứa danh sách các tin nhắn trao đổi giữa người dùng và trợ lý ảo dưới khóa `messages`.

Thông thường, chúng ta sử dụng `MessagesState` của LangGraph để làm cấu trúc mặc định. `MessagesState` được tích hợp sẵn một cơ chế đặc biệt gọi là Reducer cho khóa `messages`.

Định nghĩa: Reducer

Reducer là một hàm quy định cách thức tích hợp (merge) dữ liệu mới trả về từ một Node vào State hiện tại của hệ thống. Với danh sách tin nhắn, hành vi mặc định mong muốn là “nối tiếp (append) tin nhắn mới vào danh sách cũ”; ngược lại, với một trường thông tin như cờ tắt/mở `is_completed`, hành vi mặc định là “ghi đè (overwrite)” giá trị cũ bằng giá trị mới.

Trong `MessagesState`, khóa `messages` được định nghĩa kèm reducer mặc định là hàm `add_messages`. Khi một node trả về `{"messages": [new_msg]}`, hàm `add_messages` sẽ tự động duyệt qua danh sách tin nhắn hiện tại:

- Nếu tin nhắn mới chưa có ID hoặc ID không trùng khớp với bất kỳ tin nhắn nào trong danh sách cũ, nó sẽ được nối thêm (append) vào cuối danh sách.
- Nếu tin nhắn mới trùng ID với một tin nhắn cũ, tin nhắn cũ đó sẽ được cập nhật (update) nội dung bằng tin nhắn mới. Đây là cơ chế nền tảng giúp thực hiện cập nhật nội dung tin nhắn streaming hoặc chỉnh sửa lịch sử.

2.2. Node là hàm Python

Mỗi Node trong đồ thị thực chất chỉ là một hàm Python nhận vào trạng thái hiện tại (State) và trả về một phần dữ liệu cập nhật (được gọi là delta state dưới dạng một từ điển Python).

Mã nguồn 2.1: Hợp đồng của một model node

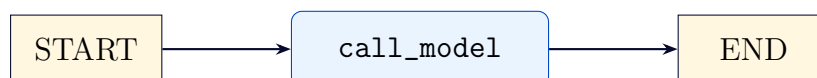
```
def call_model(state: MessagesState) -> dict:
    latest = state["messages"][-1].content
    response = make_response(latest)
    return {"messages": [response]}
```

2.2.1. Lưu ý quan trọng khi viết Node

Node không nhất thiết phải trả về toàn bộ State. Nó chỉ cần trả về phần dữ liệu thay đổi (delta) mà nó chịu trách nhiệm tạo ra. LangGraph sẽ tự động lấy dữ liệu delta này và áp dụng reducer tương ứng để cập nhật vào State chung. Thiết kế này giúp hệ thống cực kỳ dễ quan sát, cho phép ghi lại log chính xác từng bước biến đổi của dữ liệu.

2.3. Edge định nghĩa thứ tự thực thi

Edge (cạnh) định nghĩa luồng di chuyển của dữ liệu giữa các node. Đồ thị LangGraph bắt đầu từ điểm xuất phát đặc biệt tên là START và kết thúc tại điểm dừng chân END.



Hình 2.1: Graph hội thoại nhỏ nhất chưa có persistence.

Nguồn: Tác giả tự dựng theo LangGraph Graph API.

Dưới đây là ví dụ hoàn chỉnh về đồ thị tối giản có thể thực thi độc lập:

Mã nguồn 2.2: Graph chạy được không cần API key

```
"""The smallest graph: START -> call_model -> END."""

import sys

from langchain_core.messages import AIMessage
from langgraph.graph import END, START, MessagesState, StateGraph

def call_model(state: MessagesState) -> dict:
    latest = state["messages"][-1].content
    return {"messages": [AIMessage(content=f"\u0110\u00e3 nh\u01eadn: {latest}")] }

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
```

```
builder.add_edge(START, "call_model")
builder.add_edge("call_model", END)
graph = builder.compile()

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    result = graph.invoke(
        {"messages": [{"role": "user", "content": "Xin ch\u00e0o LangGraph"}]}
    )
    print(result["messages"][-1].content)
```

2.3.1. Giải thích từng dòng code trong graph_basics.py

- **Dòng 9-11:** Định nghĩa node `call_model`. Node này lấy tin nhắn cuối cùng trong danh sách `state["messages"][-1].content`, bọc nó lại bằng một tin nhắn dạng `AIMessage` kèm tiền tố “Đã nhận: ”, rồi trả về delta dạng `{"messages": [AIMessage(...)]}`.
- **Dòng 14:** Khởi tạo đồ thị `StateGraph` sử dụng cấu trúc state `MessagesState`.
- **Dòng 15:** Khai báo node `"call_model"` vào đồ thị bằng hàm `builder.add_node`.
- **Dòng 16-17:** Nối luồng thực thi từ `START` đến node `"call_model"` và từ node `"call_model"` đến `END` thông qua `builder.add_edge`.
- **Dòng 18:** Gọi phương thức `compile()` để đóng băng và kiểm tra tính hợp lệ về logic của đồ thị. Đồ thị lúc này sẵn sàng để chạy thông qua hàm `invoke`.
- **Dòng 22-23:** Đoạn `sys.stdout.reconfigure(encoding="utf-8")` đảm bảo các ký tự tiếng Việt Unicode hiển thị chính xác trên console Windows.

2.4. Compile không đồng nghĩa với Persistence

Hàm `builder.compile()` chịu trách nhiệm biên dịch đồ thị, chuyển đổi cấu trúc định nghĩa (`StateGraph`) thành một đối tượng đồ thị thực thi được. Tuy nhiên, bản thân quá trình `compile()` này không đi kèm việc lưu trữ lịch sử lâu dài.

Nếu bạn muốn đồ thị tự động ghi nhớ lịch sử hội thoại xuyên suốt nhiều lượt gọi, bạn bắt buộc phải truyền thêm một cơ chế lưu trữ checkpoint (gọi là `checkpoint`) trong lúc biên dịch đồ thị:

Mã nguồn 2.3: Cùng graph, khác năng lực lưu state

```
stateless_graph = builder.compile()
stateful_graph = builder.compile(checkpointer=checkpointer)
```

Cảnh báo: Nhầm lẫn thường gặp

`MessagesState` chỉ là cấu trúc mô tả cách thức lưu trữ dữ liệu tin nhắn trong bộ nhớ RAM tạm thời của một lượt gọi; nó hoàn toàn không có khả năng tự lưu giữ tin nhắn qua nhiều lượt gọi độc lập. Chỉ khi tích hợp `checkpointer`, LangGraph mới kích hoạt khả năng bền vững hóa dữ liệu (persistence) theo từng phiên (`thread_id`).

2.5. Đọc đồ thị bằng năm câu hỏi vàng

Mỗi khi tiếp cận hoặc thiết kế bất kỳ đồ thị LangGraph nào, hãy tự trả lời năm câu hỏi sau:

1. Cấu trúc State schema gồm có những khóa nào và kiểu dữ liệu là gì?
2. Mỗi Node thực hiện đọc dữ liệu từ khóa nào và trả về delta ở khóa nào?
3. Cạnh (edge) hay điều kiện (conditional edge) nào quyết định luồng chuyển tiếp giữa các node?
4. Đồ thị được compile kèm theo checkpointer (cho lưu trữ ngắn hạn) hay store (cho lưu trữ dài hạn) nào?
5. Khi gọi thực thi đồ thị bằng `invoke()`, cấu hình (config) và ngữ cảnh chạy (runtime context) mang những định danh (`thread_id`, `user_id`) nào?

Điểm tự kiểm tra: Lab 20 phút

Thêm một node trung gian mang tên `normalize_input` vào trước node `call_model`. Node này có nhiệm vụ chuẩn hóa văn bản của tin nhắn người dùng gửi lên (loại bỏ khoảng trắng dư thừa ở đầu và cuối văn bản). Hãy vẽ lại sơ đồ đồ thị mới, dự đoán giá trị State thay đổi qua từng bước và viết kiểm thử assert cho kết quả đầu ra.

3. SHORT-TERM MEMORY THEO THREAD

Short-term memory (bộ nhớ ngắn hạn) trong kiến trúc tác tử là vùng nhớ trạng thái được gắn chặt với một phiên hội thoại cụ thể, được xác định thông qua mã định danh luồng trò chuyện (`thread_id`). Khi một đồ thị được biên dịch (compile) kèm theo một checkpoint, LangGraph sẽ tự động lưu lại trạng thái (State) dưới dạng các điểm kiểm tra (checkpoint) sau mỗi bước thực thi của đồ thị và khôi phục lại trạng thái này ở lần gọi tiếp theo (LangChain 2026d).

3.1. Tích hợp InMemorySaver vào đồ thị

Cách đơn giản nhất để kích hoạt bộ nhớ ngắn hạn là sử dụng lớp lưu trữ tạm thời trong RAM mang tên `InMemorySaver`.

Mã nguồn 3.1: Ba dòng tạo short-term memory

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "thread-a"}}
```

Mỗi khi bạn gọi phương thức `graph.invoke(input, config)` với cùng một cấu hình chứa `thread_id`, LangGraph sẽ tự động thực hiện chu trình khép kín:

1. Truy vấn cơ sở dữ liệu checkpointer để nạp checkpoint trạng thái gần nhất tương ứng với `thread_id`.
2. Hợp nhất (merge) dữ liệu đầu vào mới của người dùng vào trạng thái vừa nạp thông qua các hàm Reducer.
3. Thực thi tuần tự các node trong đồ thị.
4. Ghi nhận trạng thái cập nhật mới nhất thành một checkpoint mới vào cơ sở dữ liệu trước khi trả về kết quả cho người dùng.

Dưới đây là ví dụ hoàn chỉnh triển khai kiểm thử độc lập hai luồng hội thoại song song:

Mã nguồn 3.2: Hai conversation độc lập bằng `InMemorySaver`

```
"""Thread-scoped short-term memory with InMemorySaver."""

import sys

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, MessagesState, StateGraph
```

```
def _find_name(messages: list[BaseMessage]) -> str | None:
    markers = ("t\u00f4i t\u00e2n l\u00e0 ", "t\u00f4i t\u00e2n ")
    for message in reversed(messages):
        if not isinstance(message, HumanMessage):
            continue
        normalized = str(message.content).strip()
        lowered = normalized.lower()
        for marker in markers:
            position = lowered.find(marker)
            if position >= 0:
                value = normalized[position + len(marker) :].strip(" .!?,")
                if value and "g\u00ec" not in value.lower():
                    return value
    return None

def deterministic_model(state: MessagesState) -> dict:
    """Replace a real LLM with rules to test memory deterministically."""
    messages = state["messages"]
    latest = str(messages[-1].content).lower()
    if "t\u00f4i t\u00e2n g\u00ec" in latest:
        name = _find_name(messages[:-1])
        answer = f"B\u00e2n t\u00e2n {name}." if name else "T\u00f4i ch\u01b0a bi\u1ebft t\u00e2n b\u00e0i"
        return {"messages": [AIMessage(content=answer)]}
    else:
        name = _find_name(messages)
        answer = f"Ch\u00e0o {name}!" if name else "T\u00f4i \u0111\u00e3 nh\u00e2n \u0111\u01b0\u0103n"
        return {"messages": [AIMessage(content=answer)]}

def build_graph(checkpointer: InMemorySaver | None = None):
    builder = StateGraph(MessagesState)
    builder.add_node("call_model", deterministic_model)
    builder.add_edge(START, "call_model")
    builder.add_edge("call_model", END)
    return builder.compile(checkpointer=checkpointer or InMemorySaver())

def ask(graph, thread_id: str, text: str) -> str:
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke(
        {"messages": [{"role": "user", "content": text}]},
        config,
    )
    return str(result["messages"][-1].content)

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    chat_graph = build_graph()
    print(ask(chat_graph, "thread-a", "T\u00f4i t\u00e2n Nh\u00e2n."))
    print(ask(chat_graph, "thread-b", "T\u00f4i t\u00e2n An."))
    print(ask(chat_graph, "thread-a", "T\u00f4i t\u00e2n g\u00ec?"))
    print(ask(chat_graph, "thread-b", "T\u00f4i t\u00e2n g\u00ec?"))
```

3.1.1. Phân tích chi tiết mã nguồn short_term_memory.py

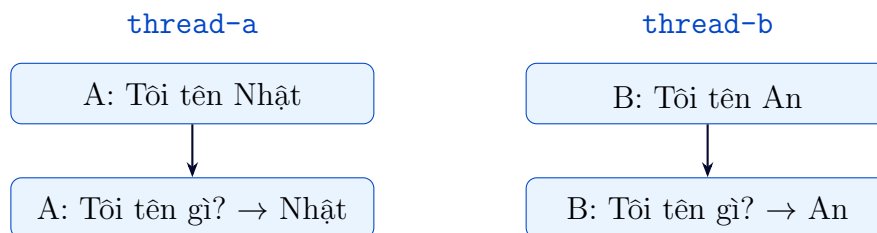
- **Hàm tiện ích _find_name:** Hàm này chịu trách nhiệm duyệt ngược danh sách tin nhắn để tìm kiếm thông tin khai báo tên từ người dùng bằng cụm từ khóa “tôi tên

là” hoặc “tôi tên”. Hàm này hoạt động độc lập với LLM API, giúp đảm bảo các kịch bản kiểm thử hành vi bộ nhớ diễn ra một cách ổn định và dễ đoán định.

- **Hàm giả lập LLM `deterministic_model`:** Hàm này đóng vai trò thay thế mô hình ngôn ngữ thực tế bằng các quy tắc logic xác định:
 - Nếu người dùng hỏi tên mình (chứa cụm từ “tôi tên gì”), hàm sẽ tìm kiếm thông tin tên trong lịch sử các tin nhắn cũ từ trước (loại trừ tin nhắn hiện tại chứa câu hỏi hỏi tên: `messages[: -1]`).
 - Nếu người dùng khai báo tên hoặc nói các câu thông thường, nó sẽ chào hoặc thông báo đã nhận được tin nhắn.
- **Hàm khởi tạo đồ thị `build_graph`:** Hàm này tạo ra một đồ thị `StateGraph` từ cấu trúc `MessagesState`, liên kết node xử lý và biên dịch đồ thị bằng việc gán checkpoint `InMemorySaver` mặc định nếu người dùng không truyền đối tượng cụ thể nào vào.
- **Hàm `ask tiện ích`:** Hàm này bọc lệnh gọi đồ thị bằng cách thiết lập cấu hình `config = {"configurable": {"thread_id": thread_id}}` để chỉ định rõ luồng hội thoại.

3.2. Dòng thời gian thực thi của hai luồng hội thoại độc lập

Sử dụng cùng một đồ thị đã biên dịch kèm theo duy nhất một đối tượng checkpoint, trạng thái của hai cuộc trò chuyện được cách ly hoàn toàn dựa vào khóa `thread_id`:



Hình 3.1: Cùng graph và checkpoint, state được phân vùng bởi thread.

Nguồn: Tác giả tự dựng.

3.3. Nguyên tắc gửi tin nhắn mới từ phía Client

Một lỗi lập trình cực kỳ phổ biến khi chuyển đổi từ kiến trúc chatbot thủ công sang sử dụng hệ thống checkpoint của LangGraph là việc client tiếp tục gửi kèm toàn bộ lịch sử trò chuyện cũ ở mỗi request:

Mã nguồn 3.3: Input đúng khi backend sở hữu history

```
graph.invoke(
```

```

{"messages": [{"role": "user", "content": new_message}]},
{"configurable": {"thread_id": thread_id}},
)

```

Cảnh báo: Tránh gửi lại lịch sử hội thoại

Nếu phía client (giao diện người dùng) vừa gửi lại toàn bộ lịch sử hội thoại vừa sử dụng cơ chế lưu checkpoint của LangGraph trên backend, reducer `add_messages` sẽ tự động nhân bản (duplicate) các tin nhắn cũ và nối thêm chúng vào trạng thái hiện tại. Lỗi này làm tăng đột biến chi phí token đầu vào mà mất thường rất khó phát hiện ra, do các giao diện UI thông thường chỉ render hiển thị một tin nhắn duy nhất một lần dựa trên ID.

3.4. Sự khác biệt giữa Thread ID và Quyền truy cập

Cảnh báo: Bảo mật Thread ID

Việc biết hoặc tạo ra một chuỗi định danh `thread_id` dạng ngẫu nhiên (UUID) hoàn toàn không tương đương với việc người dùng đó được phép truy cập cuộc trò chuyện đó. Ở môi trường production thực tế, tầng ứng dụng web (API gateway) bắt buộc phải kiểm tra thông tin người dùng được xác thực (`user_id`) có thực sự sở hữu phiên trò chuyện (`thread_id`) đó hay không trước khi thực hiện các lệnh đọc hoặc ghi vào đồ thị.

3.5. Đọc trạng thái đồ thị phục vụ Debug và Hiển thị lịch sử

Để kiểm tra trực tiếp trạng thái hiện tại của đồ thị hoặc lấy toàn bộ lịch sử tin nhắn phục vụ hiển thị trên giao diện người dùng, chúng ta có thể sử dụng phương thức `get_state`:

Mã nguồn 3.4: Đọc snapshot hiện tại của thread

```

config = {"configurable": {"thread_id": "thread-a"}}
snapshot = graph.get_state(config)

for message in snapshot.values["messages"]:
    print(message.type, message.content)

```

3.5.1. Lưu ý khi sử dụng `get_state`

Phương thức `get_state` trả về đối tượng chứa toàn bộ thông tin trạng thái tại thời điểm checkpoint hiện tại. Đối tượng này rất hữu ích cho các chức năng debug hoặc xây dựng API trả về lịch sử trò chuyện. Tuy nhiên, API không được phép trả về trực tiếp toàn bộ

thông tin thô của checkpoint ra ngoài client nếu trong đó chứa các thông tin nhạy cảm của các node nội bộ như kết quả gọi tool hay các biến trạng thái trung gian.

Điểm tự kiểm tra: Lab 25 phút

Chạy thử tệp tin `short_term_memory.py`. Hãy bổ sung thêm luồng hội thoại của người dùng C, không khai báo thông tin tên và thực hiện câu hỏi “Tôi tên gì?”. Sau đó, thử cố tình gửi yêu cầu của người dùng B nhưng lại truyền nhầm `thread_id` của cuộc trò chuyện A để quan sát hiện tượng rò rỉ dữ liệu (data leak). Hãy viết mã test tự động chứng minh sự cách ly hoàn toàn giữa 3 phiên trò chuyện.

4. QUẢN LÝ CONTEXT DÀI

Khả năng lưu trữ dữ liệu (persistence) trả lời cho câu hỏi: “Hệ thống có thể lưu giữ lại được những gì?”. Trong khi đó, quản lý ngữ cảnh (context management) trả lời cho câu hỏi: “Trong lượt gọi mô hình hiện tại, LLM thực tế được đọc những gì?”. Đây là hai bài toán hoàn toàn độc lập và cần được phân tách rõ ràng. Một phiên trò chuyện có thể đã tích lũy tới 2.000 tin nhắn để phục vụ công tác đối soát dữ liệu (audit), nhưng khi gọi LLM, chúng ta chỉ nên gửi một bản tóm tắt (summary) kèm theo khoảng 20 tin nhắn gần nhất.

4.1. Vì sao ngữ cảnh quá dài không hề miễn phí?

Mặc dù các mô hình ngôn ngữ thế hệ mới hỗ trợ kích thước ngữ cảnh (context window) lên đến hàng trăm nghìn hoặc hàng triệu token, việc gửi toàn bộ lịch sử dài vào mô hình ở mỗi request mang lại nhiều tác hại nghiêm trọng:

- **Độ trễ (Latency) tăng cao:** Thời gian xử lý mã hóa đầu vào (prompt processing) tăng tuyến tính theo độ dài văn bản gửi lên.
- **Chi phí tài chính gia tăng:** Bạn phải trả tiền cho mỗi token đầu vào ở mỗi lượt gọi. Gửi lịch sử lặp đi lặp lại sẽ làm chi phí tăng lũy tiến rất nhanh.
- **Sự phân tâm của mô hình (Lost in the Middle):** LLM có xu hướng chú ý nhiều hơn đến thông tin ở phần đầu và phần cuối của ngữ cảnh, trong khi dễ bỏ sót các chi tiết quan trọng nằm ở giữa. Thông tin cũ, lỗi thời hoặc lạc đề có thể làm giảm đáng kể độ chính xác của câu trả lời.

Để giải quyết bài toán này, tài liệu LangChain chia sẻ ba chiến lược xử lý lịch sử phổ biến: cắt bớt (trim), xóa vĩnh viễn (delete), và tóm tắt (summarize) (LangChain 2026f).

Định nghĩa: Token budget

Token budget (ngân sách token) là giới hạn số lượng token tối đa được phân bổ cho toàn bộ dữ liệu đầu vào gửi đến mô hình trong một lượt gọi, bao gồm: system instruction (chỉ dẫn hệ thống), long-term memory (bộ nhớ dài hạn), running summary, các tin nhắn gần nhất, tài liệu tham khảo RAG và kết quả của các công cụ (tool results). Không bao giờ được sử dụng 100% kích thước context window của mô hình cho phần lịch sử đầu vào, vì mô hình cần khoảng trống ngữ cảnh còn lại để sinh ra nội dung câu trả lời (output reserve).

4.2. Cắt bớt tin nhắn bằng kỹ thuật tạo View ngữ cảnh

Chúng ta có thể giới hạn số lượng tin nhắn gửi đến mô hình bằng cách tạo ra một lát cắt (view) tạm thời từ danh sách tin nhắn đầy đủ mà không làm ảnh hưởng đến lịch sử lưu trữ gốc trong cơ sở dữ liệu.

Mã nguồn 4.1: Trim bản input, không xóa storage

```
"""Long-context strategies: trimming and a simple running summary."""

from dataclasses import dataclass

from langchain_core.messages import BaseMessage
from langchain_core.messages.utils import (
    count_tokens_approximately,
    trim_messages,
)

def context_for_model(
    messages: list[BaseMessage],
    *,
    max_tokens: int = 2_000,
) -> list[BaseMessage]:
    """Create a short model view without deleting stored history."""
    return trim_messages(
        messages,
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=max_tokens,
        start_on="human",
        end_on=("human", "tool", "ai"),
        include_system=True,
    )
```

4.2.1. Phân tích chi tiết hàm context_for_model

Hàm sử dụng phương thức `trim_messages` của LangChain để lọc tin nhắn dựa trên ngân sách token:

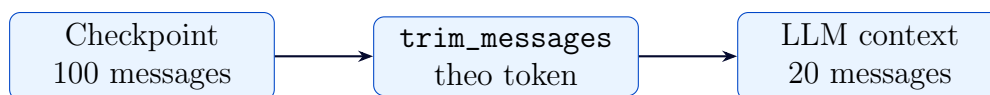
- **strategy="last":** Chỉ định cắt từ đầu danh sách và giữ lại các tin nhắn ở cuối danh sách (các tin nhắn gần đây nhất).
- **token_counter=count_tokens_approximately:** Hàm đếm số lượng token ước tính dựa trên độ dài chuỗi ký tự (hoặc có thể cấu hình bằng tokenizer thực tế của mô hình như tiktoken).
- **max_tokens=max_tokens:** Ngân sách token tối đa cho phép của View ngữ cảnh.
- **start_on="human":** Đây là tham số cực kỳ quan trọng. Nó đảm bảo ngữ cảnh gửi vào mô hình luôn luôn bắt đầu bằng một tin nhắn từ phía người dùng (`HumanMessage`). Nếu phần cắt bớt vô tình cắt đúng vào tin nhắn của trợ lý (`AIMessage`) hoặc kết quả của công cụ (`ToolMessage`) ở đầu danh sách, hàm sẽ tự động dịch chuyển điểm

cắt để bắt đầu ở tin nhắn người dùng tiếp theo, tránh làm mô hình bối rối về mặt cấu trúc hội thoại.

- **include_system=True:** Đảm bảo tin nhắn cấu hình hệ thống (**SystemMessage**) định nghĩa vai trò trợ lý sẽ luôn luôn được giữ lại ở đầu danh sách ngữ cảnh gửi đi, không bị ảnh hưởng bởi quá trình cắt bớt.

4.3. Không cắt tin nhắn một cách mù quáng

Cắt danh sách tin nhắn bằng cách lấy lát cắt thô kiểu Python như `messages[-20:]` là một cách làm rất nguy hiểm. Nếu trong 20 tin nhắn cuối đó chứa một tin nhắn trợ lý yêu cầu gọi công cụ (`tool_call`) nhưng tin nhắn trả về kết quả công cụ (`ToolMessage`) tương ứng lại nằm ở tin nhắn thứ 21 (đã bị cắt đi), API của các nhà cung cấp như OpenAI hay Anthropic sẽ lập tức từ chối xử lý và báo lỗi giao thức hội thoại không hợp lệ. Do đó, việc sử dụng các hàm tiện ích có hiểu biết về loại tin nhắn như `trim_messages` là bắt buộc.



Hình 4.1: Trim tạo view cho model nhưng có thể giữ storage đầy đủ.

Nguồn: Tác giả tự dựng.

Cảnh báo: Phân biệt hai kiểu xóa tin nhắn

Cần phân biệt rõ:

1. **Cắt bớt ở mức Input (Trim View):** Chỉ ẩn các tin nhắn cũ đi đối với mô hình trong lượt gọi hiện tại, dữ liệu trong cơ sở dữ liệu checkpoint vẫn được bảo toàn nguyên vẹn.
2. **Xóa vĩnh viễn ở mức State (State Deletion):** Trả về đối tượng `RemoveMessage(id=msg_id)` từ node. LangGraph sẽ lập tức đánh dấu và xóa vĩnh viễn tin nhắn đó khỏi cơ sở dữ liệu lưu trữ ở lần ghi checkpoint tiếp theo. Hãy chỉ sử dụng cơ chế này khi hệ thống có chính sách bảo mật dữ liệu nghiêm ngặt hoặc khi người dùng yêu cầu xóa lịch sử.

4.4. Mẫu phân bổ ngân sách ngữ cảnh hợp lý

Bảng dưới đây minh họa một phương án phân bổ ngân sách ngữ cảnh (token budget) thực tế cho một mô hình hỗ trợ ngữ cảnh đầu vào 32.000 token:

Bảng 4.1: Ví dụ phân bổ ngữ cảnh ngữ âm (Token Budget)

Thành phần ngữ cảnh	Ngân sách (Token)	Ghi chú
System instruction và chính sách (Policy)	2.000	Luôn luôn được giữ lại nguyên vẹn ở đầu.
Long-term user profile (Truy xuất từ Store)	1.000	Chỉ nạp các thông tin ghi nhớ có liên quan.
Running summary (Tóm tắt hội thoại cũ)	3.000	Phần tóm tắt cô đọng của các tin nhắn đã bị cắt.
Recent messages (Các lượt chat gần đây nhất)	10.000	Giữ nguyên văn khoảng 5-10 cặp hội thoại gần nhất.
RAG / Kết quả công cụ hỗ trợ	8.000	Ngữ cảnh mở rộng khi truy xuất cơ sở dữ liệu.
Output reserve (Ngân sách dự phòng đầu ra)	8.000	Phần trống dành riêng cho mô hình sinh câu trả lời.

Điểm tự kiểm tra: Lab 20 phút

Tạo ra một danh sách giả lập gồm 30 tin nhắn hội thoại xen kẽ, đưa một từ khóa bí mật (fact) vào tin nhắn thứ 3. Thực hiện chạy thử hàm `context_for_model` với ba cấu hình giới hạn token khác nhau. Ghi nhận số lượng tin nhắn thực tế còn lại trong View ngữ cảnh đầu ra và kiểm tra xem từ khóa bí mật đó có bị mất đi khi thu hẹp ngân sách token hay không. Viết báo cáo phân tích giải pháp khắc phục. nd học viên tự thực hành và ghi nhận.

5. RUNNING SUMMARY (TÓM TẮT TĂNG DẦN)

Trong khi kỹ thuật cắt bớt tin nhắn (trimming) hoạt động rất nhanh và tiết kiệm tài nguyên, nhược điểm lớn nhất của nó là làm biến mất vĩnh viễn các dữ kiện (facts) quan trọng xuất hiện ở phần đầu cuộc trò chuyện. Để khắc phục triệt để điều này, chúng ta sử dụng kỹ thuật Running Summary (tóm tắt tăng dần hay lũy tiến).

Phương pháp này nén toàn bộ các tin nhắn cũ đã bị cắt đi thành một đoạn văn bản tóm tắt súc tích lưu trong State, sau đó gửi kèm đoạn tóm tắt này cùng với các tin nhắn gần nhất đến mô hình LLM ở các lượt gọi sau. Cả LangChain và LangGraph đều hỗ trợ xây dựng cơ chế này một cách linh hoạt thông qua các node tự định nghĩa (LangChain 2026a,f).

5.1. Mở rộng State Schema để chứa Summary

Để tích hợp bộ nhớ tóm tắt, bước đầu tiên là chúng ta cần kế thừa cấu trúc `MessagesState` mặc định và khai báo thêm trường `summary`:

Mã nguồn 5.1: State có thêm summary

```
from langgraph.graph import MessagesState

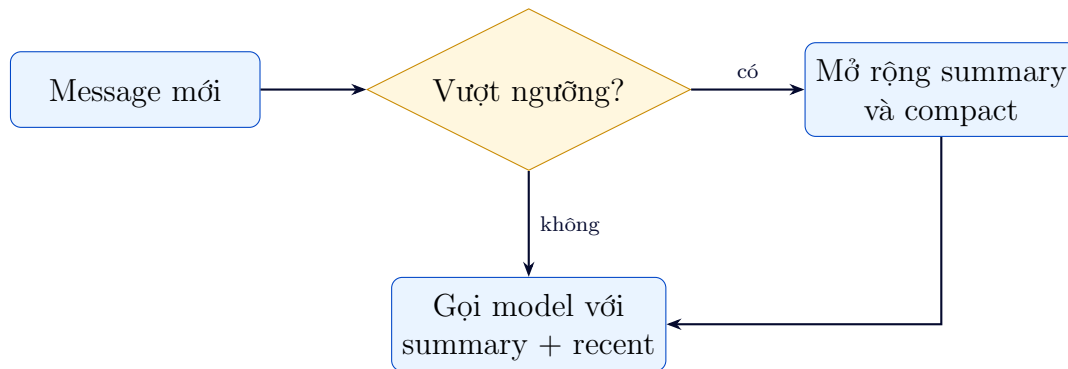
class ChatState(MessagesState):
    summary: str
```

5.1.1. Bản chất của trường dữ liệu summary

Cần lưu ý rằng biến `summary` định nghĩa ở trên vẫn là một phần của bộ nhớ ngắn hạn (short-term memory) vì nó được gắn liền với trạng thái của một luồng trò chuyện (`thread_id`) cụ thể. Nó được lưu trữ và cập nhật tự động bởi checkpointer. Nó hoàn toàn không tự chuyển hóa thành hồ sơ người dùng dài hạn (long-term profile) nếu chúng ta không có cơ chế lưu trữ riêng biệt.

5.2. Chu trình hoạt động của Node tóm tắt lũy tiến

Chu trình xử lý tóm tắt lịch sử hội thoại chỉ nên được kích hoạt khi độ dài của danh sách tin nhắn hiện tại vượt quá một ngưỡng giới hạn nhất định (ví dụ: vượt quá 10 tin nhắn hoặc vượt quá 2.000 token):



Hình 5.1: Running summary chỉ chạy khi history vượt ngưỡng.

Nguồn: Tác giả tự dựng.

Quy trình thực thi của Node tóm tắt (Summarizer node) diễn ra như sau:

1. Ở lượt chạy đầu tiên vượt ngưỡng: Đọc toàn bộ danh sách tin nhắn hiện tại, yêu cầu LLM sinh ra một đoạn tóm tắt tổng hợp lần thứ nhất, sau đó thay thế các tin nhắn cũ bằng các đối tượng `RemoveMessage` để giải phóng ngữ cảnh.
2. Ở các lượt chạy tiếp theo: Summarizer node nhận vào đoạn **summary** cũ đã lưu trong State kèm theo các tin nhắn mới phát sinh (chưa được nén). Nó yêu cầu LLM viết tiếp một bản tóm tắt mới tích hợp cả hai nguồn thông tin trên, tạo ra một bản tóm tắt cập nhật (lũy tiến) duy nhất.

5.3. Tiêu chí của một bản tóm tắt chất lượng cao

Khi viết chỉ dẫn hệ thống (prompt) cho Summarizer node, chúng ta cần hướng dẫn mô hình tập trung giữ lại các nhóm thông tin cốt lõi sau:

- Mục tiêu hiện tại và các ràng buộc nghiệp vụ mà người dùng đã đề cập.
- Các quyết định quan trọng đã được hai bên thống nhất và lý do đi kèm.
- Các giải pháp kỹ thuật đã thử nghiệm, kết quả thu được hoặc mã lỗi nghiêm trọng phát sinh.
- Tên của các thực thể (entities), mã định danh (IDs) nghiệp vụ cần thiết cho các bước xử lý sau.
- Các câu hỏi còn bỏ ngỏ hoặc các hành động tiếp theo (action items) chưa được giải quyết.

5.3.1. Thông tin cần lược bỏ

Cần yêu cầu mô hình chủ động loại bỏ các câu chào hỏi xã giao, câu từ lặp lại vô nghĩa, các giải thích dài dòng đã hết giá trị hiệu lực. Do tóm tắt là quá trình suy diễn (inference),

mô hình hoàn toàn có khả năng viết sai hoặc bịa đặt thông tin (hallucination). Với các quyết định nghiệp vụ hoặc dữ liệu đặc thù nhạy cảm, nên thiết kế cấu trúc lưu trữ chuyên biệt (structured memory) thay vì chỉ lưu trữ dưới dạng một chuỗi văn bản thuần túy.

5.4. Mô phỏng cơ chế Compact tin nhắn trong mã nguồn

Để hiểu rõ hơn về mặt cấu trúc dữ liệu, chúng ta hãy xem xét hàm mô phỏng thu gọn tin nhắn bằng rule trong `context_management.py`:

Mã nguồn 5.2: Mô phỏng compact không cần LLM

```
class CompactedConversation:
    summary: str
    recent_messages: list[dict[str, str]]

def compact_deterministically(
    messages: list[dict[str, str]],
    *,
    keep_recent: int = 4,
) -> CompactedConversation:
    """Demonstrate compaction with rules; this is not an LLM summarizer."""
    if len(messages) <= keep_recent:
        return CompactedConversation("", messages.copy())

    old_messages = messages[:-keep_recent]
    facts: list[str] = []
    for message in old_messages:
        content = message["content"].strip()
        if message["role"] == "user" and content:
            facts.append(f"User said: {content}")
    summary = " ".join(facts[-5:])
    return CompactedConversation(summary, messages[-keep_recent:])
```

5.4.1. Phân tích chi tiết mã nguồn

- **Định nghĩa lớp CompactedConversation:** Một dataclass chứa hai trường: `summary` (chuỗi văn bản tóm tắt) và `recent_messages` (danh sách các tin nhắn gần nhất được giữ lại nguyên văn).
- **Hàm compact_deterministically:** Hàm này mô phỏng quá trình tóm tắt:
 - Nếu tổng số tin nhắn nhỏ hơn hoặc bằng tham số `keep_recent` (mặc định là 4), hàm trả về danh sách tin nhắn hiện tại mà không tóm tắt.
 - Nếu vượt ngưỡng, hàm phân tách danh sách tin nhắn thành hai phần: `old_messages` (phần tin nhắn cũ cần tóm tắt) và phần tin nhắn gần đây. Nó duyệt qua `old_messages`, lọc ra các câu nói của người dùng và nối lại thành một chuỗi văn bản đại diện cho bộ nhớ tóm tắt.

Trong môi trường thực tế, đoạn logic mô phỏng trên được thay thế hoàn toàn bằng việc

gọi một mô hình LLM nhỏ hơn (để tiết kiệm chi phí) kết hợp với cấu trúc đầu ra chuẩn hóa (Structured Output) dạng JSON để dễ dàng phân tích và kiểm soát chất lượng:

Mã nguồn 5.3: Schema summary để đánh giá

```
summary = {  
  "goal": "Integrate LangGraph into the LLM Gateway",  
  "decisions": ["Use one thread_id per conversation"],  
  "facts": ["User name is Nhat"],  
  "open_questions": ["Choose SQLite or PostgreSQL"]  
}
```

5.5. Lựa chọn thời điểm Trigger chạy tóm tắt

Có ba phương án chính để kích hoạt tiến trình tóm tắt hội thoại:

- **Theo số lượng tin nhắn (Message Count):** Dễ lập trình nhất nhưng không phản ánh chính xác độ dài thực tế (ví dụ: 5 tin nhắn cực dài có thể chứa lượng token nhiều hơn 20 tin nhắn cực ngắn).
- **Theo số lượng token thực tế (Token Threshold):** Sát với giới hạn của mô hình nhất, giúp tối ưu hóa không gian ngữ cảnh hiệu quả nhất.
- **Chạy ngầm ở luồng nền (Asynchronous Background Job):** Giúp triệt tiêu hoàn toàn độ trễ của tiến trình tóm tắt trên đường chạy chính (hot path) của người dùng, nhưng đòi hỏi thiết kế hệ thống phức tạp để tránh xung đột ghi dữ liệu (race conditions).

Đối với các dự án thử nghiệm (prototype), việc kích hoạt tóm tắt theo token ngay trước bước gọi mô hình chính là phương án tối ưu nhất về mặt vận hành và độ phức tạp.

Điểm tự kiểm tra: Lab 25 phút

Thiết kế một kịch bản hội thoại mẫu gồm 12 tin nhắn có chứa thông tin khai báo tên, mục tiêu dự án, một quyết định nghiệp vụ bị thay đổi ở giữa cuộc trò chuyện và một câu hỏi chưa trả lời. Hãy viết một cấu trúc prompt yêu cầu LLM tóm tắt thông tin theo đúng định dạng JSON 4 trường ở trên. Viết mã kiểm thử tự động xác thực:

1. Bản tóm tắt có giữ lại đúng quyết định mới nhất hay không?
2. Bản tóm tắt có loại bỏ quyết định cũ đã hết hiệu lực hay không?
3. Bản tóm tắt có sinh ra các dữ kiện ảo (hallucination) hay không?

6. LONG-TERM MEMORY THEO USER

Bộ nhớ dài hạn (Long-term memory) giải quyết bài toán lưu giữ các thông tin ổn định của người dùng xuyên suốt qua nhiều cuộc trò chuyện (thread) khác nhau. Trong kiến trúc LangGraph, bộ nhớ dài hạn được tổ chức dưới dạng các tài liệu JSON lưu trữ trong một cấu trúc dữ liệu chuyên biệt gọi là Store. Mỗi tài liệu trong Store được định vị chính xác thông qua một không gian tên (namespace) phân cấp và một khóa (key) định danh (LangChain 2026e).

Để xây dựng hệ thống bộ nhớ chính xác, nhà phát triển cần phân biệt rõ ràng hai thành phần lưu trữ độc lập sau:

Bảng 6.1: So sánh giữa Checkpointer và Store

	Checkpoint (Bộ nhớ ngắn hạn)	Store (Bộ nhớ dài hạn)
Phạm vi hoạt động	Giới hạn trong phạm vi một luồng chat (<code>thread_id</code>).	Toàn cục, hoạt động xuyên suốt tất cả các luồng chat của người dùng.
Khóa truy xuất	Sử dụng <code>thread_id</code> .	Namespace phân cấp chứa <code>user_id</code> .
Dữ liệu lưu trữ	Toàn bộ danh sách tin nhắn thô, running summary, trạng thái thực thi hiện tại.	Hồ sơ người dùng (profile), tùy chọn cá nhân (preferences), các quy tắc hành vi, thông tin chọn lọc.
Ví dụ điển hình	Nội dung người dùng vừa nói ở câu chat ngay trước đó.	Người dùng yêu cầu trợ lý luôn viết mã nguồn giải thích bằng tiếng Việt.

6.1. Thiết kế cấu trúc Namespace và Key

Store lưu trữ dữ liệu dưới dạng sơ đồ đường dẫn logic (namespace), giúp chúng ta dễ dàng tổ chức phân cấp thư mục dữ liệu theo cơ chế multi-tenant hoặc phân vùng theo từng người dùng.

Mã nguồn 6.1: Profile key-value trong InMemoryStore

```
"""Key-value long-term memory namespaced by user_id."""

import sys
from typing import Any
```

```
from langgraph.store.memory import InMemoryStore

def profile_namespace(user_id: str) -> tuple[str, str, str]:
    return ("users", user_id, "profile")

def save_profile(store: InMemoryStore, user_id: str, profile: dict[str, Any]) -> None:
    store.put(profile_namespace(user_id), "main", profile)

def load_profile(store: InMemoryStore, user_id: str) -> dict[str, Any]:
    item = store.get(profile_namespace(user_id), "main")
    return dict(item.value) if item else {}

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    memory_store = InMemoryStore()
    save_profile(
        memory_store,
        "user-nhat",
        {
            "name": "Nh\u1eadt",
            "language": "Vietnamese",
            "answer_style": "t\u1eebng b\u01b0\u1edbc",
        },
    )
    print(load_profile(memory_store, "user-nhat"))
```

6.1.1. Phân tích chi tiết mã nguồn long_term_memory.py

- **Namespace phân cấp profile_namespace:** Hàm trả về một tuple dạng ("users", user_id, "profile"). Đây là quy ước thiết kế namespace cực kỳ quan trọng trong LangGraph. Việc đưa user_id vào vị trí thứ hai giúp chúng ta dễ dàng thực hiện phân quyền truy cập và tìm kiếm dữ liệu của một người dùng cụ thể mà không lo sợ bị chồng lấn dữ liệu giữa các tài khoản khác nhau.
- **Phương thức ghi dữ liệu save_profile:** Sử dụng hàm store.put để ghi đè hoặc ghi mới tài liệu JSON vào namespace tương ứng dưới tên khóa chính là "main".
- **Phương thức đọc dữ liệu load_profile:** Sử dụng hàm store.get để lấy tài liệu từ Store. Nếu tài liệu tồn tại, chúng ta truy cập thuộc tính item.value để lấy nội dung từ điển JSON gốc, ngược lại trả về một từ điển rỗng.

6.2. Phân loại bộ nhớ dài hạn đáng lưu trữ

Không phải mọi thông tin người dùng nói ra đều cần được đưa vào bộ nhớ dài hạn. Việc lưu trữ quá mức sẽ làm nhiễu ngữ cảnh và vi phạm các quyền riêng tư. Chúng ta chia bộ nhớ dài hạn làm ba nhóm chính:

1. **Semantic memory (Bộ nhớ ngữ nghĩa):** Lưu trữ các dữ kiện ổn định về thực

tế như tên người dùng, ngôn ngữ giao tiếp ưa thích, cấu hình môi trường phát triển của họ.

2. **Episodic memory (Bộ nhớ trải nghiệm):** Lưu giữ các sự kiện hoặc kinh nghiệm quan trọng đã xảy ra trong quá khứ có ích cho tương lai (ví dụ: người dùng báo hệ thống đang dùng thư viện Docker phiên bản cũ).
3. **Procedural memory (Bộ nhớ quy trình):** Các quy tắc ứng xử mà người dùng đặt ra cho tác tử (ví dụ: luôn đưa ra mã nguồn mẫu trước rồi mới giải thích chi tiết lý thuyết sau).

Cảnh báo: Chính sách bảo mật thông tin nhạy cảm

Tuyệt đối không ghi các thông tin nhạy cảm như API key, mã truy cập (tokens), mật khẩu tài khoản, thông tin thẻ thanh toán vào Store dưới dạng văn bản thô. Hệ thống cần được thiết kế đi kèm các tính năng cho phép người dùng chủ động xem lại toàn bộ thông tin tác tử đang nhớ, cập nhật hoặc yêu cầu xóa bỏ hoàn toàn (quyền được quên) các thông tin đó.

6.3. Giải pháp kết hợp Thread và User trong một Dịch vụ hội thoại

Để tạo ra trải nghiệm người dùng tối ưu, chúng ta kết hợp đồng thời Checkpointer và Store trong một dịch vụ duy nhất mang tên ChatService.

Mã nguồn 6.2: ChatService kết hợp checkpointer và Store

```
"""Combine a short-term checkpointer and long-term Store."""

from dataclasses import dataclass
from typing import Any

from langchain_core.messages import AIMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, MessagesState, StateGraph
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore

from long_term_memory import load_profile, save_profile
from short_term_memory import _find_name

@dataclass(frozen=True)
class UserContext:
    user_id: str

class ChatService:
    def __init__(
        self,
        *,
        checkpointer: InMemorySaver | None = None,
        store: InMemoryStore | None = None,
    ) -> None:
        self.checkpointer = checkpointer or InMemorySaver()
        self.store = store or InMemoryStore()
```

```
def call_model(state: MessagesState, runtime: Runtime[UserContext]) -> dict:
    user_id = runtime.context.user_id
    messages = state["messages"]
    latest = str(messages[-1].content)
    lowered = latest.lower()

    if "h\u00e3y nh\u01b0\u0300" in lowered and (name := _find_name(messages)):
        profile = load_profile(self.store, user_id)
        profile["name"] = name
        save_profile(self.store, user_id, profile)
        answer = f"T\u00f4i s\u01b0\u0300 nh\u01b0\u0300 b\u01b0\u0300 t\u00e2n {name}."
    elif "t\u00f4i t\u00e2n g\u00ec" in lowered:
        thread_name = _find_name(messages[:-1])
        profile_name = load_profile(self.store, user_id).get("name")
        name = thread_name or profile_name
        answer = f"B\u01b0\u0300 t\u00e2n {name}." if name else "T\u00f4i ch\u01b0\u0300a bi\u0300 t\u00e2n
        b\u01b0\u0300."
    else:
        answer = "T\u00f4i \u0111\u00e3 nh\u01b0\u0300 \u0111\u01b0\u0300\u01b0\u0300\u01b0\u0300 tin nh\u01b0\u0300."

    return {"messages": [AIMessage(content=answer)]}

builder = StateGraph(MessagesState, context_schema=UserContext)
builder.add_node("call_model", call_model)
builder.add_edge(START, "call_model")
builder.add_edge("call_model", END)
self.graph = builder.compile(
    checkpointer=self.checkpointer,
    store=self.store,
)

def ask(self, *, user_id: str, thread_id: str, message: str) -> str:
    config = {"configurable": {"thread_id": thread_id}}
    result = self.graph.invoke(
        {"messages": [{"role": "user", "content": message}]},
        config,
        context=UserContext(user_id=user_id),
    )
    return str(result["messages"][-1].content)

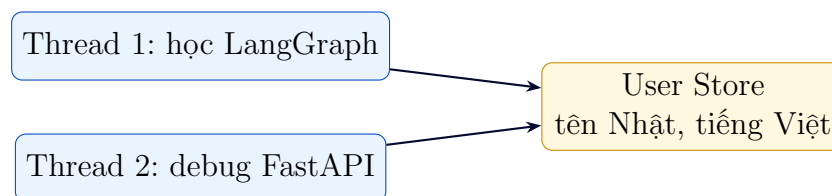
def history(self, thread_id: str) -> list[dict[str, Any]]:
    config = {"configurable": {"thread_id": thread_id}}
    snapshot = self.graph.get_state(config)
    return [
        {"role": message.type, "content": str(message.content)}
        for message in snapshot.values.get("messages", [])
    ]

def delete_thread(self, thread_id: str) -> None:
    self.checkpointer.delete_thread(thread_id)
```

6.3.1. Phân tích chi tiết luồng xử lý trong ChatService

- Truyền ngữ cảnh thông qua Runtime[UserContext]:** LangGraph cung cấp cơ chế Runtime để truyền các tham số môi trường chạy (như `user_id`) vào các node mà không cần phải nhồi nhét chúng vào danh sách tin nhắn hoặc cấu hình đồ thị. Bằng cách định nghĩa `context_schema=UserContext` trong `StateGraph`, node `call_model` có thể dễ dàng truy xuất thông tin người dùng hiện tại thông qua biến `runtime.context.user_id`.

- **Kịch bản ghi nhớ chủ động (Opt-in Memory):** Khi người dùng gửi tin nhắn chứa cụm từ “hãy nhớ” kèm theo việc khai báo tên, node xử lý sẽ trích xuất tên thông qua hàm `_find_name`, nạp thông tin profile hiện tại từ Store lên, cập nhật trường `"name"` và lưu ngược trở lại Store thông qua hàm `save_profile`.
- **Phân cấp truy xuất bộ nhớ (Memory Hierarchy Fallback):** Khi người dùng hỏi “Tôi tên gì?”, hệ thống sẽ kiểm tra theo thứ tự ưu tiên:
 1. Tìm kiếm thông tin tên trong lịch sử hội thoại ngắn hạn của chính luồng đó thông qua `_find_name(messages[:-1])`.
 2. Nếu luồng chat mới được khởi tạo và chưa có thông tin tên, hệ thống tiếp tục truy vấn bộ nhớ dài hạn trong Store thông qua `load_profile(self.store, user_id)`.
 3. Nếu cả hai nguồn lưu trữ đều không có thông tin, hệ thống mới đưa ra câu trả lời mặc định là chưa biết tên.



Hình 6.1: Nhiều thread của cùng user truy cập một namespace memory.

Nguồn: Tác giả tự dựng.

Điểm tự kiểm tra: Lab 25 phút

Viết và chạy một bộ kiểm thử tự động giả lập hai kịch bản sau:

1. Người dùng A ở luồng hội thoại 1 yêu cầu trợ lý “Hãy nhớ tôi tên Nhật”. Sau đó, khởi tạo luồng hội thoại 2 cho người dùng A và hỏi trợ lý xem tên mình là gì. Kết quả mong muốn là trợ lý phải nhớ được tên Nhật.
2. Người dùng B ở luồng hội thoại 3 thực hiện câu hỏi hỏi tên. Kết quả mong muốn là trợ lý trả lời chưa biết tên (đảm bảo tính cách ly dữ liệu tuyệt đối giữa người dùng A và người dùng B).

7. PERSISTENCE VÀ RANH GIỚI PRODUCTION

Lớp lưu trữ tạm thời trong RAM (`InMemorySaver`) giúp kiểm chứng nhanh các lỗi logic trong lúc phát triển, nhưng không đảm bảo tính bền vững của dữ liệu trên môi trường thực tế. Khi thiết kế hệ thống chạy trên môi trường production, nhóm phát triển bắt buộc phải làm rõ lộ trình lưu trữ dữ liệu hội thoại, cơ chế đồng bộ, mã hóa, sao lưu, và kiểm soát quyền truy cập của người dùng.

7.1. Chọn lựa cơ sở dữ liệu lưu trữ theo từng giai đoạn

LangGraph tách biệt các triển khai lưu trữ checkpointer thành các thư viện Python độc lập. Tùy thuộc vào quy mô và yêu cầu vận hành của hệ thống, chúng ta lựa chọn cơ sở dữ liệu phù hợp:

Bảng 7.1: Lộ trình backend cho bộ nhớ tác tử

Cơ sở dữ liệu	Quy mô phù hợp	Giới hạn hoặc yêu cầu vận hành
InMemorySaver / Store	Viết Unit test, workshop ngắn hạn.	Dữ liệu mất sạch khi khởi động lại tiến trình; không thể chia sẻ dữ liệu giữa các worker.
SQLite (SqliteSaver)	Thiết bị cá nhân, ứng dụng demo, thử nghiệm bền vững local.	Ghi dữ liệu vào một file duy nhất; cần quản lý cẩn thận tranh chấp kết nối (connection lock).
PostgreSQL (PostgresSaver)	Ứng dụng web, hệ thống doanh nghiệp (Enterprise).	Đòi hỏi thiết lập hạ tầng DB, quản lý pool kết nối, sao lưu định kỳ, và giám sát hiệu năng.
Redis / MongoDB	Kiến trúc doanh nghiệp đã có sẵn hạ tầng tương ứng.	Chỉ lựa chọn khi hạ tầng chung yêu cầu, tránh đưa thêm công nghệ mới không thiết thực.

7.2. Xử lý bất đồng bộ khi tích hợp Cơ sở dữ liệu

Hầu hết các framework web hiện đại như FastAPI hoạt động dựa trên cơ chế bất đồng bộ (asynchronous event loop). Nếu đồ thị của bạn thực hiện các tác vụ gọi mô hình hoặc gọi API bên thứ ba bất đồng bộ (`ainvoke`), bạn bắt buộc phải sử dụng các saver bất đồng bộ tương ứng như `AsyncSqliteSaver` hoặc `AsyncPostgresSaver`.

Việc gọi các câu lệnh đồng bộ (blocking I/O) của database trên event loop sẽ gây tắc nghẽn toàn bộ hệ thống FastAPI, làm giảm hiệu năng nghiêm trọng. Hãy thiết lập kết

nổi database trong Lifespan của FastAPI và đóng chúng lại khi ứng dụng tắt:

Mã nguồn 7.1: Khung lifecycle cho checkpointer production

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from langgraph.checkpoint.postgres.ai import AsyncPostgresSaver

DB_URI = "postgresql://user:pass@localhost:5432/chat_db"

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Khởi tạo connection pool và checkpointer bắt đầu bo
    async with AsyncPostgresSaver.from_conn_string(DB_URI) as saver:
        # Tự động tạo các bảng cần thiết nếu chạy lần đầu
        await saver.setup()
        app.state.chat_graph = build_graph(checkpointer=saver)
    yield
    # Connection tự động đóng khi thoát khỏi block async with
```

7.3. Ranh giới Phân quyền sở hữu dữ liệu (Ownership)

Mặc dù LangGraph checkpointer tự động nhóm và quản lý trạng thái tin nhắn dựa vào khóa `thread_id`, bản thân checkpointer không có nhận thức về mặt bảo mật hay phân quyền sở hữu. Nhiệm vụ bảo vệ ranh giới dữ liệu thuộc về lớp ứng dụng web (API Gateway / FastAPI):

1. **Xác thực (Authentication):** Trích xuất token xác thực của người dùng gửi lên qua header để nhận diện `user_id` tin cậy.
2. **Phân quyền (Authorization):** Tra cứu trong cơ sở dữ liệu xem luồng hội thoại `thread_id` hiện tại có thực sự thuộc quyền sở hữu của `user_id` này hay không.
3. **Thực thi an toàn:** Chỉ khi xác nhận quyền sở hữu trùng khớp, ứng dụng mới cho phép đọc trạng thái (`get_state`) hoặc gọi đồ thị (`invoke`).

Ví dụ: Thiết kế bảng quản lý hội thoại tối thiểu

Để quản lý ranh giới dữ liệu, chúng ta thiết kế bảng dữ liệu quan hệ lưu trữ thông tin hội thoại như sau:

```
CREATE TABLE conversations (
    thread_id UUID PRIMARY KEY,
    owner_user_id UUID NOT NULL,
    title TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);
```

7.4. Quy định về thời hạn lưu trữ (Retention) và Xóa dữ liệu

Khi người dùng nhấn nút “Xóa lịch sử trò chuyện” trên giao diện, ứng dụng không chỉ ẩn tin nhắn đó trên màn hình frontend. Một quy trình xóa dữ liệu chuẩn xác phải thực hiện xóa thông tin checkpoint vật lý lưu trong database để bảo vệ quyền riêng tư của người dùng:

- Sử dụng phương thức xóa chuyên biệt của checkpointer để giải phóng dữ liệu checkpoint tương ứng với `thread_id`.
- Cập nhật cột `deleted_at` trong bảng quản lý hội thoại (Soft Delete) để ẩn hội thoại khỏi danh sách hiển thị của người dùng, trước khi tiến hành xóa cứng (Hard Delete) bằng các tiến trình định kỳ (Cron Jobs).
- Phân biệt rõ giữa việc xóa luồng hội thoại (`thread_id`) và xóa bộ nhớ dài hạn của người dùng lưu trong Store. Việc xóa một cuộc trò chuyện ngắn hạn không được phép làm mất đi các tùy chọn cá nhân dài hạn đã ghi trong Store, trừ khi người dùng chủ động yêu cầu thông qua tính năng “Quên thông tin của tôi”.

Điểm tự kiểm tra: Bài thiết kế 20 phút

Hãy so sánh hai kiến trúc lưu trữ sau đây đối với hai dự án thực tế:

1. Chatbot hỗ trợ viết code chạy local trên máy tính cá nhân của một nhà phát triển.
2. Hệ thống chatbot CSKH phục vụ 10.000 người dùng đồng thời chạy trên nền tảng Cloud (Kubernetes).

Hãy phân tích và đưa ra ít nhất 3 lý do lựa chọn loại cơ sở dữ liệu lưu trữ checkpoint, 1 rủi ro tiềm ẩn và phương án thử nghiệm khôi phục dữ liệu sau khi xảy ra sự cố restart hệ thống cho mỗi dự án.

8. TÍCH HỢP VÀO LLM GATEWAY

Dự án đích thực tế của chúng ta bao gồm giao diện Streamlit, dịch vụ web FastAPI, và sự hỗ trợ của hai nhà cung cấp mô hình dịch vụ (providers) lớn là DeepSeek và Gemini. Thay đổi mang tính bước ngoặt khi đưa bộ nhớ vào hệ thống không đơn thuần chỉ là thêm một lệnh import thư viện LangGraph. Đó là sự chuyển đổi toàn diện hợp đồng dịch vụ từ cơ chế “gửi prompt đơn lẻ” sang cơ chế “gửi tin nhắn mới trong một phiên trò chuyện (conversation) được kiểm soát quyền sở hữu (owner)”.

8.1. Hợp đồng API đề xuất cho cổng dịch vụ (LLM Gateway)

Để đảm bảo tính độc lập và bảo mật, chúng ta đề xuất bộ khung tài nguyên API gồm các endpoint cơ bản sau:

Mã nguồn 8.1: Tạo conversation và gửi message

```
# 1. End point tạo phiên trò chuyện mới
POST /conversations
{"user_id": "user-nhat"}

Response: 201 Created
{"thread_id": "c20f...", "user_id": "user-nhat"}

# 2. End point gửi tin nhắn mới vào phiên trò chuyện
POST /conversations/c20f.../messages
{
  "user_id": "user-nhat",
  "message": "Ten toi la gi?"
}

Response: 200 OK
{"thread_id": "c20f...", "answer": "Ban ten la Nhat."}
```

8.1.1. Luồng dữ liệu giữa Client và Server

Giao diện người dùng (Streamlit frontend) chịu trách nhiệm lưu trữ định danh luồng hiện tại (**thread_id**). Khi người dùng tạo cuộc trò chuyện mới, frontend sẽ gọi endpoint khởi tạo để nhận về một **thread_id** dạng ngẫu nhiên (UUID). Ở các lượt chat tiếp theo, frontend chỉ gửi duy nhất tin nhắn mới nhất của người dùng kèm theo **thread_id**; tuyệt đối không gửi kèm các tin nhắn cũ vì backend và checkpointer đã chịu trách nhiệm lưu giữ lịch sử.

8.2. Xây dựng API hoàn chỉnh bằng FastAPI

Dưới đây là mã nguồn dịch vụ FastAPI hoàn chỉnh tích hợp bộ nhớ hội thoại LangGraph:

Mã nguồn 8.2: FastAPI conversation resource

```
"""Minimal conversation API with a per-thread ownership check."""

from uuid import uuid4

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

from chat_service import ChatService

app = FastAPI(title="LangGraph Memory Lab")
chat_service = ChatService()
thread_owners: dict[str, str] = {}

class CreateConversationRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)

class MessageRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)
    message: str = Field(min_length=1, max_length=100_000)

def require_owner(thread_id: str, user_id: str) -> None:
    owner = thread_owners.get(thread_id)
    if owner is None:
        raise HTTPException(status_code=404, detail="Conversation not found.")
    if owner != user_id:
        raise HTTPException(status_code=403, detail="Conversation owner mismatch.")

@app.post("/conversations", status_code=201)
def create_conversation(payload: CreateConversationRequest):
    thread_id = str(uuid4())
    thread_owners[thread_id] = payload.user_id
    return {"thread_id": thread_id, "user_id": payload.user_id}

@app.post("/conversations/{thread_id}/messages")
def send_message(thread_id: str, payload: MessageRequest):
    require_owner(thread_id, payload.user_id)
    answer = chat_service.ask(
        user_id=payload.user_id,
        thread_id=thread_id,
        message=payload.message.strip(),
    )
    return {"thread_id": thread_id, "answer": answer}

@app.get("/conversations/{thread_id}/messages")
def get_messages(thread_id: str, user_id: str):
    require_owner(thread_id, user_id)
    return {"thread_id": thread_id, "messages": chat_service.history(thread_id)}

@app.delete("/conversations/{thread_id}", status_code=204)
def delete_conversation(thread_id: str, user_id: str):
    require_owner(thread_id, user_id)
    chat_service.delete_thread(thread_id)
```

```
del thread_owners[thread_id]
```

8.2.1. Phân tích chi tiết mã nguồn fastapi_app.py

- **Cấu trúc kiểm tra quyền sở hữu require_owner:** Hàm này lấy `thread_id` và `user_id` làm đối số. Nó truy vấn từ điển `thread_owners` (đóng vai trò giả lập bảng CSDL hội thoại):
 - Nếu `thread_id` chưa từng được khởi tạo, hàm trả về lỗi HTTP 404 (Not Found).
 - Nếu cuộc trò chuyện tồn tại nhưng thuộc quyền sở hữu của người dùng khác, hàm lập tức trả về lỗi HTTP 403 (Forbidden).
- **Phương thức create_conversation:** Khi người dùng gửi yêu cầu tạo luồng chat kèm theo định danh `user_id`, hàm sinh một mã UUID ngẫu nhiên làm `thread_id`, đăng ký quyền sở hữu vào từ điển `thread_owners` và trả lại thông tin cho client.
- **Phương thức gửi tin nhắn send_message:** Hàm thực hiện kiểm tra quyền sở hữu trước thông qua `require_owner`, sau đó gọi phương thức `chat_service.ask` để thực thi đồ thị LangGraph và nhận về câu trả lời.
- **Phương thức xóa hội thoại delete_conversation:** Giải phóng toàn bộ dữ liệu checkpoint ngắn hạn trong checkpointer thông qua `chat_service.delete_thread` và xóa quyền sở hữu trong từ điển `thread_owners`.

8.3. Adapter tích hợp mô hình thực tế (DeepSeek và Gemini)

Để ứng dụng dễ dàng chuyển đổi qua lại giữa hai nhà cung cấp dịch vụ DeepSeek và Gemini mà không làm xáo trộn cấu trúc đồ thị, chúng ta sử dụng các lớp bọc (wrappers) chuẩn hóa tin nhắn của LangChain:

Mã nguồn 8.3: Adapter DeepSeek và Gemini

```
"""Real model adapters. This module needs the 'providers' extra and API keys."""

import os

from langchain_core.language_models.chat_models import BaseChatModel

def build_model(provider: str) -> BaseChatModel:
    if provider == "deepseek":
        from langchain_openai import ChatOpenAI

        return ChatOpenAI(
            api_key=os.environ["DEEPSEEK_API"],
            base_url=os.getenv("BASE_URL", "https://api.deepseek.com"),
            model=os.getenv("DEEPSEEK_MODEL", "deepseek-chat"),
            max_tokens=int(os.getenv("MAX_OUTPUT_TOKENS", "4096")),
```

```

    )

    if provider == "gemini":
        from langchain_google_genai import ChatGoogleGenerativeAI

        return ChatGoogleGenerativeAI(
            google_api_key=os.environ["GEMINI_API_KEY"],
            model=os.getenv("GEMINI_MODEL_NAME", "gemini-2.5-flash"),
        )

    raise ValueError(f"Unsupported provider: {provider}")

```

8.3.1. Phân tích chi tiết mã nguồn real_model_adapter.py

- **Bọc mô hình DeepSeek:** Sử dụng lớp ChatOpenAI từ gói langchain_openai. Nhờ tính tương thích cao của API DeepSeek với chuẩn OpenAI, chúng ta chỉ cần chuyển đổi địa chỉ kết nối base_url thành https://api.deepseek.com và truyền API key tương ứng.
- **Bọc mô hình Gemini:** Sử dụng lớp ChatGoogleGenerativeAI từ gói langchain_google_genai và truyền mã khóa API Google.

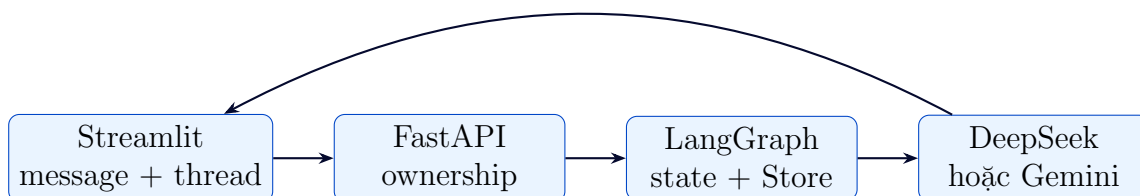
Khi có các adapter trên, Node gọi mô hình trong đồ thị sẽ cực kỳ ngắn gọn và độc lập với nhà cung cấp:

Mã nguồn 8.4: Thay model quyết định bằng chat model thật

```

async def call_model(state: MessagesState):
    # Cut old messages based on token budget
    model_messages = context_for_model(
        state["messages"],
        max_tokens=8_000,
    )
    # Call model asynchronously via ainvoke
    response = await model.ainvoke(model_messages)
    return {"messages": [response]}

```



Hình 8.1: Ranh giới thành phần trong LLM Gateway có memory.

Nguồn: Tác giả tự dựng.

8.4. Lộ trình di cư mã nguồn (Migration Slices)

Để tích hợp hệ thống bộ nhớ vào một dự án Gateway lớn đang chạy ổn định một cách an toàn nhất, chúng ta cần chia nhỏ quy trình thành các lát cắt nhỏ (thin slices):

1. **Lát 1:** Giữ nguyên mô hình giả (rule-based mock model); tích hợp cổng FastAPI để kiểm thử cô lập tính cách ly và bảo mật giữa các luồng.
2. **Lát 2:** Thay thế mô hình giả bằng mô hình thực thông qua adapter bọc; chưa tích hợp cơ chế tóm tắt (summary) hay cắt bớt (trimming).
3. **Lát 3:** Tích hợp hàm cắt bớt tin nhắn `trim_messages` và các metric đo đặc token đầu vào/đầu ra.
4. **Lát 4:** Thay thế lưu trữ tạm thời trong RAM bằng lưu trữ SQLite hoặc PostgreSQL thực tế; kiểm thử kịch bản tắt/mở server.
5. **Lát 5:** Bổ sung tính năng lưu trữ tùy chọn cá nhân dài hạn (Store) khi người dùng chủ động cho phép.

Cảnh báo: Tránh tích hợp mọi tính năng cùng một lúc

Nếu bạn đồng thời thay đổi cấu hình cổng API, thay thế adapter gọi mô hình, cấu hình thêm database lưu trữ checkpoint, và bật tính năng tóm tắt cùng một lúc, khi xảy ra lỗi (như chatbot trả lời sai hoặc mất lịch sử), bạn sẽ hoàn toàn không thể định vị được nguyên nhân nằm ở tầng ngữ cảnh (prompt), tầng lưu trữ (persistence), hay lỗi logic của mô hình. Mỗi bước di cư cần được đi kèm với các bài viết kiểm thử tự động tương ứng.

Điểm tự kiểm tra: Lab dự án 30 phút

Khởi chạy dịch vụ FastAPI local bằng lệnh: `uv run uvicorn fastapi_app:app -app-dir code -reload`. Sử dụng tài liệu tương tác Swagger API để:

1. Khởi tạo hai cuộc trò chuyện độc lập của người dùng A và người dùng B.
2. Thực hiện gửi tin nhắn khai báo tên khác nhau vào mỗi cuộc trò chuyện.
3. Thử nghiệm dùng tài khoản của người dùng B để gửi yêu cầu đọc lịch sử tin nhắn của người dùng A. Kết quả mong muốn: Mã trạng thái lỗi bảo mật HTTP 403 trả về thành công trước khi hệ thống thực hiện đọc checkpoint.

9. KIỂM THỬ, DEBUG VÀ QUAN SÁT

Một chatbot hoạt động theo kiểu “thỉnh thoảng trả lời có vẻ nhớ” hoàn toàn không đủ tiêu chuẩn để đưa lên môi trường chạy thực tế. Chúng ta cần thiết lập các bài kiểm thử tự động để bảo đảm các luật bất biến (invariants) về bộ nhớ được duy trì xuyên suốt các đợt cập nhật tính năng. Nguyên tắc cốt lõi là việc kiểm thử bộ nhớ phải được tách biệt hoàn toàn khỏi sự bất định (non-determinism) của LLM bằng cách sử dụng các mô hình giả lập hoạt động theo luật cố định (rule-based mock models).

9.1. Bốn quy tắc bất biến cốt lõi về bộ nhớ

Mọi hệ thống bộ nhớ chatbot đều phải vượt qua bốn quy tắc kiểm thử sau:

1. **Cùng luồng (Same Thread):** Tin nhắn sau bắt buộc phải đọc được thông tin ngữ cảnh của tin nhắn trước.
2. **Khác luồng (Different Thread):** Trạng thái ngắn hạn (short-term memory) tuyệt đối không được phép rò rỉ qua lại giữa hai luồng trò chuyện khác nhau.
3. **Cùng người dùng, khác luồng (Same User, Cross Thread):** Bộ nhớ dài hạn (long-term memory) trong Store phải có khả năng chia sẻ xuyên suốt các luồng của cùng một tài khoản.
4. **Khác người dùng (Different User):** Bộ nhớ dài hạn của người dùng A tuyệt đối không được phép rò rỉ sang người dùng B.

Dưới đây là mã nguồn kiểm thử tự động bốn quy tắc bất biến trên sử dụng thư viện `pytest`:

Mã nguồn 9.1: Test short-term và long-term isolation

```
from uuid import uuid4

from langgraph.store.memory import InMemoryStore

from chat_service import ChatService
from long_term_memory import load_profile, save_profile
from short_term_memory import ask, build_graph

def test_two_threads_do_not_mix_names():
    graph = build_graph()
    thread_a = str(uuid4())
    thread_b = str(uuid4())

    ask(graph, thread_a, "T\u00f4i t\u00ean Nh\u1eadt.")
    ask(graph, thread_b, "T\u00f4i t\u00ean An.")
```

```

assert ask(graph, thread_a, "T\u00f4i t\u00ean g\u00ec?" ) == "B\u00ean t\u00ean Nh\u00e0t."
assert ask(graph, thread_b, "T\u00f4i t\u00ean g\u00ec?" ) == "B\u00ean t\u00ean An."

def test_profile_is_separated_by_user_id():
    store = InMemoryStore()
    save_profile(store, "user-a", {"name": "Nh\u00e0t"})
    save_profile(store, "user-b", {"name": "An"})

    assert load_profile(store, "user-a")["name"] == "Nh\u00e0t"
    assert load_profile(store, "user-b")["name"] == "An"

def test_long_term_memory_crosses_threads_for_same_user():
    service = ChatService()
    service.ask(
        user_id="user-nhat",
        thread_id=str(uuid4()),
        message="H\u00e0i nh\u00e0t t\u00e0i t\u00e0i l\u00e0 Nh\u00e0t.",
    )

    answer = service.ask(
        user_id="user-nhat",
        thread_id=str(uuid4()),
        message="T\u00f4i t\u00ean g\u00ec?",
    )
    assert answer == "B\u00ean t\u00ean Nh\u00e0t."

def test_long_term_memory_does_not_cross_users():
    service = ChatService()
    service.ask(
        user_id="user-a",
        thread_id=str(uuid4()),
        message="H\u00e0i nh\u00e0t t\u00e0i t\u00e0i l\u00e0 Nh\u00e0t.",
    )

    answer = service.ask(
        user_id="user-b",
        thread_id=str(uuid4()),
        message="T\u00f4i t\u00ean g\u00ec?",
    )
    assert answer == "T\u00f4i ch\u00e0 bi\u00e0t t\u00e0i b\u00e0i."

```

9.1.1. Phân tích chi tiết mã nguồn test_memory.py

- **Hàm test_two_threads_do_not_mix_names:** Khởi tạo đồ thị bằng mô hình luật xác định, tạo hai luồng ngẫu nhiên bằng UUID. Người dùng A nói tên Nhật ở luồng A, người dùng B nói tên An ở luồng B. Hàm assert kiểm tra kết quả trả về của từng luồng độc lập có chính xác hay không.
- **Hàm test_profile_is_separated_by_user_id:** Lưu trữ thông tin trực tiếp vào Store dưới hai tài khoản khác nhau (user-a và user-b) và thực hiện assert kiểm tra sự cách ly dữ liệu ở tầng Store vật lý.
- **Hàm test_long_term_memory_crosses_threads_for_same_user:** Khởi tạo dịch vụ ChatService. Gửi câu lệnh ghi nhớ ở một luồng chat ngẫu nhiên của người dùng Nhật, sau đó thực hiện câu hỏi hỏi tên ở một luồng chat mới tinh của cùng người

dùng đó. Kết quả assert chứng minh bộ nhớ dài hạn đã hoạt động xuyên luồng hội thoại.

- **Hàm `test_long_term_memory_does_not_cross_users`:** Tương tự như trên nhưng câu hỏi được thực hiện bởi tài khoản `user-b`. Kết quả assert chứng minh tài khoản B nhận được câu trả lời mặc định là chưa biết tên, không bị ảnh hưởng bởi dữ liệu của tài khoản A.

9.2. Kiểm thử tầng API, bảo vệ ranh giới bảo mật

Đồ thị LangGraph chạy độc lập có thể đúng về mặt logic bộ nhớ, nhưng khi đóng gói vào API của FastAPI, hệ thống vẫn có khả năng bị rò rỉ thông tin nếu các lập trình viên quên thực hiện việc kiểm tra quyền sở hữu tại các hàm route. Do đó, viết kiểm thử tích hợp (integration tests) cho API bằng `TestClient` là cực kỳ quan trọng:

Mã nguồn 9.2: Test ownership tại ranh giới FastAPI

```
from fastapi.testclient import TestClient

from fastapi_app import app

def test_conversation_flow_and_ownership():
    with TestClient(app) as client:
        created = client.post("/conversations", json={"user_id": "user-a"})
        assert created.status_code == 201
        thread_id = created.json()["thread_id"]

        response = client.post(
            f"/conversations/{thread_id}/messages",
            json={"user_id": "user-a", "message": "T\u00f4i t\u00ean Nh\u00e0t."},
        )
        assert response.status_code == 200

        forbidden = client.get(
            f"/conversations/{thread_id}/messages",
            params={"user_id": "user-b"},
        )
        assert forbidden.status_code == 403

def test_unknown_thread_returns_404():
    with TestClient(app) as client:
        response = client.get(
            "/conversations/not-found/messages",
            params={"user_id": "user-a"},
        )
        assert response.status_code == 404
```

9.2.1. Phân tích chi tiết mã nguồn `test_fastapi_app.py`

- **Hàm `test_conversation_flow_and_ownership`:** Giả lập một client gọi endpoint POST tạo hội thoại cho `user-a`. Sau đó, giả lập một client đại diện cho tài khoản khác (`user-b`) cố tình gọi endpoint GET để đọc danh sách tin nhắn của hội thoại

vừa tạo. Kết quả assert kiểm tra mã trạng thái trả về có đúng là lỗi HTTP 403 (Forbidden) hay không.

- **Hàm `test_unknown_thread_returns_404`:** Kiểm tra hành vi hệ thống khi người dùng gọi một `thread_id` không tồn tại trên hệ thống. Kết quả assert xác nhận lỗi HTTP 404 (Not Found) được trả về chính xác.

9.3. Failure Lab: Các lỗi kinh điển và cách xử lý

Hãy chủ động làm quen với các tình huống lỗi sau trong lúc xây dựng hệ thống:

- **Sử dụng thread ID tĩnh (Static Thread ID):** Gán cứng một giá trị `thread_id` trong code client. Triệu chứng: Mọi người dùng trên hệ thống đều nhìn thấy tin nhắn của nhau và phản hồi của trợ lý bị lẫn lộn hoàn toàn.
- **Gửi trùng lặp lịch sử từ Client (Duplicate History):** Client vừa gửi lịch sử hội thoại vừa sử dụng checkpoint. Triệu chứng: Số lượng token đầu vào tăng vọt nhanh chóng, câu trả lời của mô hình lặp đi lặp lại do reducer nhận đôi tin nhắn cũ.
- **Lưu trữ tạm thời bị mất dữ liệu (Volatile Storage):** Sử dụng saver dạng RAM tạm thời trong môi trường Docker container. Triệu chứng: Dữ liệu hội thoại biến mất hoàn toàn mỗi khi container bị crash hoặc cập nhật phiên bản mới.
- **Namespace Store thiếu `user_id` (Missing User Namespace):** Lưu trữ hồ sơ người dùng vào Store mà không phân vùng namespace theo `user_id`. Triệu chứng: Người dùng đăng nhập sau ghi đè tên của người dùng đăng nhập trước trên toàn hệ thống.

9.4. Giám sát Vận hành (Telemetry & Metrics)

Khi đưa hệ thống bộ nhớ vào vận hành thực tế, chúng ta cần theo dõi sát sao các chỉ số (metrics) sau để đảm bảo hệ thống chạy mượt mà và tối ưu hóa chi phí:

Bảng 9.1: Các chỉ số giám sát vận hành bộ nhớ tác tử

Metric cần đo đạc	Mục đích giám sát	Cảnh báo cần lưu ý
Mã băm (hash) của thread và user ID	Tra vết luồng yêu cầu (traceability)	Tránh log các thông tin thô dạng PII của người dùng.
Số lượng tin nhắn trong checkpoint	Phát hiện lỗi nhân đôi lịch sử trò chuyện	Số lượng tin nhắn tăng bất thường sau mỗi lượt chat.
Tổng số token đầu vào/đầu ra (mỗi provider)	Kiểm soát chi phí tài chính	Token đầu vào vượt quá Token budget đã thiết lập.
Độ trễ đọc/ghi checkpoint và Store	Đánh giá hiệu năng database	Độ trễ tăng cao do nghẽn kết nối DB hoặc thiếu index.
Tần suất chạy tóm tắt (summary trigger)	Tối ưu hóa hiệu năng gọi LLM	Tóm tắt chạy liên tục sau mỗi lượt gọi (tốn token).
Số lượng mã lỗi bảo mật HTTP 403	Cảnh báo an ninh/lỗi giao diện client	Tần suất lỗi 403 đột biến báo hiệu hành vi tấn công dò quét.

Điểm tự kiểm tra: Lab 20 phút

Hãy chạy toàn bộ các bài test tự động bằng lệnh `uv run pytest`. Sau đó, thử cố tình loại bỏ tham số `user_id` khỏi tuple namespace trong file `long_term_memory.py` rồi chạy lại `pytest`. Hãy quan sát xem bài test nào bị thất bại và giải thích cơ chế phát hiện lỗi bảo mật của bài test đó.

10. DỰ ÁN CUỐI KHÓA: CHATBOT NHIỀU CUỘC HỘI THOẠI

10.1. Mô tả đề tài và Mục tiêu học tập

Dự án Capstone cuối khóa đóng vai trò tổng hợp toàn bộ các kỹ năng thiết kế hệ thống hội thoại có bộ nhớ. Học viên được yêu cầu nâng cấp toàn diện hệ thống cổng dịch vụ (LLM Gateway) viết bằng FastAPI kết hợp giao diện Streamlit nhằm đạt được các mục tiêu kỹ thuật thực tế: cho phép một người dùng tạo và quản lý song song nhiều phiên hội thoại độc lập, đảm bảo lịch sử trò chuyện được lưu trữ bền vững qua các lần khởi động lại máy chủ, kiểm soát chặt chẽ kích thước ngữ cảnh gửi đi, và chỉ lưu thông tin bộ nhớ dài hạn khi người dùng cho phép rõ ràng.

10.2. Yêu cầu chức năng chi tiết

Học viên phải hoàn thành tối thiểu các tính năng chức năng sau đây:

- Quản lý phiên hội thoại (Thread Management):** Giao diện người dùng (Streamlit) hỗ trợ các tính năng tạo luồng chat mới, hiển thị danh sách các luồng chat cũ, cho phép đổi tên tiêu đề luồng chat và xóa vĩnh viễn luồng chat.
- Tương tác không trạng thái phía Client:** Client gửi tin nhắn mới lên server chỉ truyền kèm `thread_id` và `user_id`; tuyệt đối không gửi kèm các tin nhắn cũ trong payload.
- Khôi phục lịch sử sau khi làm mới giao diện:** Khi người dùng nhấn F5 làm mới trình duyệt, frontend tự động truy vấn endpoint lấy lịch sử tin nhắn để vẽ lại toàn bộ các bong bóng chat cũ một cách chính xác.
- Không trộn lẫn trạng thái giữa các Nhà cung cấp (Providers):** Cho phép người dùng chuyển đổi qua lại giữa DeepSeek và Gemini tại giao diện cấu hình runtime mà không làm mất hoặc trộn lẫn lịch sử tin nhắn của luồng hiện tại.
- Cắt bớt tin nhắn tự động (Auto-trimming):** Tích hợp cơ chế tự động tính toán token và cắt bớt tin nhắn đầu vào theo token budget trước khi gọi LLM API.
- Tóm tắt lũy tiến (Running Summary):** Tự động kích hoạt node tóm tắt lịch sử cũ khi tổng kích thước tin nhắn vượt quá ngưỡng token cấu hình.
- Ghi nhớ chủ động (Opt-in Long-term Memory):** Trợ lý chỉ được phép ghi nhận các tùy chọn cá nhân hoặc dữ kiện dài hạn vào Store khi nhận được câu lệnh

bắt đầu bằng từ khóa “Hãy nhớ ...”. Trợ lý phải hỗ trợ câu lệnh “Hãy quên ...” để thực hiện xóa bỏ thông tin tương ứng.

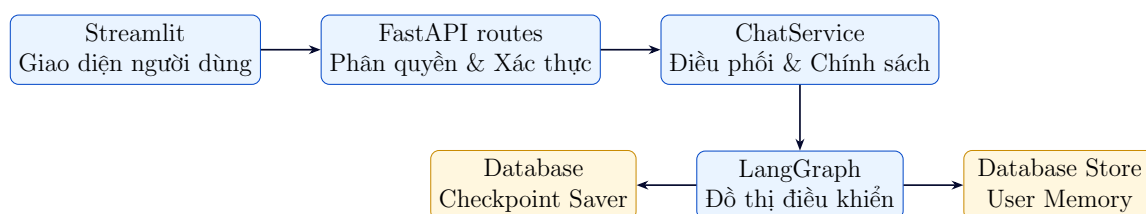
8. **Bảo mật cách ly người dùng tuyệt đối:** Ngăn chặn triệt để hành vi rò rỉ dữ liệu chéo (cross-user data leak). Người dùng A tuyệt đối không được phép đọc, ghi hoặc xóa luồng chat hay bộ nhớ Store thuộc sở hữu của người dùng B.

10.3. Yêu cầu phi chức năng và Vận hành

- **An toàn thông tin (Security):** Tuyệt đối không lưu các khóa bí mật (API Keys), mật khẩu CSDL hoặc tokens xác thực dưới dạng văn bản thô trong mã nguồn hoặc log. Cấu hình tất cả thông qua biến môi trường.
- **Xử lý ngoại lệ (Exception Handling):** Thiết lập cơ chế timeout (ví dụ: 15 giây) và tự động thử lại (retry) khi gọi LLM API. Trả về thông báo lỗi có nghĩa ngoài giao diện thay vì làm crash tiến trình backend.
- **Độ bền dữ liệu (Durability):** Tích hợp SQLite hoặc PostgreSQL làm checkpointer vật lý lưu trữ checkpoint, chứng minh dữ liệu khôi phục hoàn chỉnh sau khi khởi động lại backend.
- **Giám sát hiệu năng (Observability):** In ra màn hình log hoặc lưu trữ metric về số lượng token tiêu thụ cho mỗi request và thời gian phản hồi (latency) của từng node trong đồ thị.
- **Tài liệu kỹ thuật (Documentation):** Viết tệp README mô tả chi tiết luồng di chuyển của dữ liệu (data flow) từ Streamlit qua FastAPI đến LangGraph, sơ đồ database, và chính sách thời hạn lưu giữ dữ liệu (retention policy).

10.4. Kiến trúc tham chiếu hệ thống Capstone

Sơ đồ dưới đây mô tả sự tương tác giữa các thành phần trong kiến trúc tham chiếu mà học viên cần triển khai:



Hình 10.1: Kiến trúc tham chiếu cho dự án cuối khóa.

Nguồn: Tác giả tự dựng.

10.5. Kế hoạch thực hiện theo mốc kiểm chứng

Bảng 10.1: Các mốc dự án có thể kiểm chứng

Mốc	Phạm vi kiểm thử	Điều kiện nghiệm thu (Definition of Ready)
1	API Conversation	Tạo thành công hai luồng chat bằng mô hình giả lập thông qua Swagger; kiểm thử phân quyền 403 hoạt động chính xác.
2	Adapter gọi mô hình thực	Kết nối thành công mô hình DeepSeek/Gemini; xử lý tốt lỗi mất kết nối mạng hoặc lỗi định dạng tin nhắn.
3	Lưu trữ vật lý bền vững	Thay thế saver bằng SQLite/PostgreSQL; restart server và kiểm chứng lịch sử không bị mất.
4	Tối ưu hóa Context	Cấu hình trim tin nhắn theo token budget; kiểm thử thành công kịch bản chat siêu dài (trên 50 tin nhắn).
5	Bộ nhớ dài hạn Store	Người dùng nói “Hãy nhớ tôi thích Python”; kiểm tra tài khoản đó được nhớ ở luồng chat mới; kiểm tra tài khoản khác không thấy thông tin này.
6	Hoàn thiện UI và Release	Giao diện Streamlit vẽ mượt mà luồng trò chuyện, hiển thị thông tin đo đạc token và thời gian phản hồi ở mỗi câu trả lời.

10.6. Kịch bản Demo 5 phút bắt buộc

Khi nghiệm thu dự án, học viên phải thực hiện trình diễn trực tiếp theo đúng 6 bước kịch bản sau:

- Bước 1 (Khai báo thông tin):** Tại luồng hội thoại A, nói với trợ lý “Tôi tên Nhật”.
- Bước 2 (Kiểm tra cách ly luồng ngắn hạn):** Khởi tạo luồng hội thoại B và hỏi “Tôi tên gì?”. Trợ lý phải trả lời chưa biết tên (chứng minh short-term memory cách ly).
- Bước 3 (Ghi nhớ dài hạn chủ động):** Quay lại luồng hội thoại A, nói “Hãy nhớ tôi tên Nhật”. Trợ lý phản hồi xác nhận sẽ ghi nhớ.
- Bước 4 (Kiểm tra chia sẻ bộ nhớ dài hạn):** Khởi tạo luồng hội thoại C mới hoàn toàn của cùng người dùng đó, thực hiện câu hỏi “Tôi tên gì?”. Trợ lý phải trả lời tên Nhật (chứng minh bộ nhớ dài hạn Store hoạt động xuyên luồng).
- Bước 5 (Kiểm tra cách ly tài khoản người dùng):** Đăng nhập bằng tài khoản người dùng B (hoặc gửi request bằng API Client giả lập người dùng B) và thực hiện câu hỏi hỏi tên. Trợ lý tuyệt đối không được phép trả lời tên Nhật.

6. **Bước 6 (Kiểm tra độ bền lưu trữ):** Thực hiện lệnh restart dịch vụ backend FastAPI. F5 lại giao diện người dùng Streamlit và chứng minh lịch sử trò chuyện vẫn được nạp và hiển thị đầy đủ.

10.7. Định nghĩa hoàn thành (Definition of Done)

Một dự án Capstone được tính là hoàn thành xuất sắc khi và chỉ khi:

- Tất cả các mốc kiểm chứng từ 1 đến 6 hoạt động ổn định và vượt qua buổi trình diễn demo 5 phút trực tiếp.
- Bộ mã nguồn có độ bao phủ kiểm thử tự động (test coverage) đạt trên 80%, vượt qua toàn bộ các bài test của pytest.
- Hệ thống không chứa bất kỳ mã khóa API hay mật khẩu thô nào trong mã nguồn.
- Có tệp tài liệu README chi tiết và chính sách quản lý thời gian lưu trữ dữ liệu rõ ràng.

Điểm tự kiểm tra: Tự phản biện thiết kế hệ thống

Học viên cần tự viết một đoạn phản biện trả lời câu hỏi: Tại sao chúng ta nên sử dụng mô hình quản lý bộ nhớ của LangGraph thay vì tự viết các câu lệnh truy vấn SQL thủ công để lưu lịch sử vào database? Những phần việc nào của bộ nhớ vẫn thuộc về trách nhiệm xử lý của tầng ứng dụng? Nếu bỏ LangGraph, bạn sẽ gặp khó khăn gì khi triển khai các quy trình tác tử phức tạp hơn (ví dụ như Human-in-the-loop)?

Kết luận

Bộ nhớ hội thoại không bắt đầu ở vector database. Nó bắt đầu ở một hợp đồng đơn giản: mỗi message thuộc đúng conversation; mỗi conversation thuộc đúng user; state được lưu có chủ đích; model chỉ đọc lát context phù hợp. LangGraph cung cấp state graph, checkpointing và Store để biểu diễn hợp đồng ấy, nhưng authorization, retention, schema memory và đánh giá chất lượng vẫn là trách nhiệm của ứng dụng.

Lộ trình an toàn là: Python thuần để hiểu cơ chế; model quyết định để test; InMemorySaver để học thread; database để có durability; trim trước summary; key-value trước semantic memory; provider thật sau khi isolation đã xanh. Khi từng lát có test, việc thêm RAG, tool hoặc agent về sau không làm mất nền móng hội thoại.

Điểm tự kiểm tra: Ba câu phải trả lời được

1. Dữ liệu nào đang được lưu, ở đâu và theo khóa nào?
 2. Lướt gọi model này thực sự nhận những message nào?
 3. Điều gì chứng minh user khác không đọc được state hoặc memory này?
- Nếu ba câu đều có câu trả lời và test quan sát được, hệ thống đã đi đúng hướng.

A. Cài đặt và lệnh chạy

A.1. Cài môi trường

Từ thư mục giáo trình:

```
uv sync --extra dev
uv run pytest -q
```

Chạy từng ví dụ:

```
uv run python code/manual_history.py
uv run python code/graph_basics.py
uv run python code/short_term_memory.py
uv run python code/long_term_memory.py
```

Chạy FastAPI lab:

```
uv run uvicorn fastapi_app:app --app-dir code --reload
```

Mở <http://127.0.0.1:8000/docs> để chạy request bằng Swagger UI.

A.2. Provider thật

```
uv sync --extra dev --extra providers
```

Đặt API key bằng biến môi trường hoặc file `.env` đã bị ignore; không commit secret. Adapter tham khảo nằm ở `code/real_model_adapter.py`.

A.3. Build PDF

Máy có MiKTeX theo cấu hình repository chạy:

```
.\build-latex.ps1
```

Output ở `build/main.pdf`. Build script dùng thư mục MiKTeX riêng của project và không ghi đè các biến profile phổ biến.

A.4. Fallback

Nếu chưa cài được LangGraph, học chương 1 bằng `manual_history.py`; nếu không có API key, chạy toàn bộ test với model quyết định; nếu chưa có PostgreSQL, dùng InMemorySaver để hoàn thiện logic rồi viết test contract cho backend sau.

B. Bài tập tổng hợp và rubric

B.1. Bài tập cấp 1: mô hình tinh thần

1. Vẽ ba lớp UI history, persistent history và model context cho một request.
2. Viết conversation manager bằng list/dictionary cho ba thread.
3. Giải thích bằng 100 từ vì sao model không tự nhớ request trước.

B.2. Bài tập cấp 2: LangGraph memory

4. Mở rộng state có `summary` và `selected_provider`.
5. Viết test 20 lượt chat xen kẽ giữa hai thread.
6. Trim theo token và đo số message model nhận.
7. Thiết kế namespace profile cho tổ chức, user và application.

B.3. Bài tập cấp 3: production

8. Thay InMemorySaver bằng SQLite và chứng minh restart.
9. Viết bảng conversation ownership và endpoint 403.
10. Thêm lệnh xem/quên long-term memory.
11. Viết kế hoạch migration lên PostgreSQL nhiều worker.

B.4. Rubric dự án cuối khóa

Bảng B.1: Rubric 100 điểm cho chatbot có memory

Tiêu chí	Điểm	Minh chứng
Mô hình state/thread/user đúng	15	Sơ đồ, schema và giải thích nhất quán.
Short-term isolation	15	Test nhiều thread, không duplicate message.
Context management	15	Token budget, trim/summary và test dài.
Long-term memory	15	Opt-in save/forget, namespace đúng.
Security và privacy	15	Ownership, secret policy, cross-user test.
Persistence và vận hành	10	Restart, lifecycle, error handling.
Kiểm thử và quan sát	10	Unit/API/integration test, metric có nghĩa.
Tài liệu và demo	5	README, sơ đồ, demo tái lập được.

B.5. Thang phản hồi bốn mức

- **1 – Chưa đạt:** chỉ demo thử công, không tách đúng thread/user.
- **2 – Đang phát triển:** flow chính chạy nhưng thiếu failure test hoặc persistence.
- **3 – Đạt:** các invariant chính có test, context và ownership rõ.
- **4 – Vững:** có đánh giá dài hạn, quan sát, migration và trade-off được chứng minh.

Mẫu phản hồi: một điểm mạnh quan sát được; một khoảng trống gần rubric; một bước sửa cụ thể; một test nhỏ phải hoàn thành trước buổi sau.

C. Thuật ngữ nhanh

Bảng C.1: Thuật ngữ cốt lõi

Thuật ngữ	Nghĩa trong giáo trình
Context window	Giới hạn token model có thể nhận trong một lượt.
Message	Đơn vị hội thoại có role và content.
State	Dữ liệu hiện tại được node đọc và cập nhật.
Node	Hàm hoặc runnable thực hiện một bước của graph.
Edge	Quan hệ điều hướng giữa các node.
Reducer	Quy tắc hợp nhất node output vào state.
Thread	Chuỗi các lượt thuộc cùng một conversation.
<code>thread_id</code>	Định danh tra state/checkpoint của conversation.
Checkpoint	Snapshot state tại một bước thực thi.
Checkpointing	Backend ghi và đọc checkpoint.
Short-term memory	State trong phạm vi một thread.
Store	Kho tài liệu JSON dùng xuyên thread.
Namespace	Tuple phân vùng item trong Store.
Long-term memory	Thông tin user/app sống qua nhiều thread.
Trim	Chọn lát message ngắn hơn cho model input.
Running summary	Bản tóm tắt tăng dần của phần hội thoại cũ.
RAG	Truy xuất kiến thức ngoài tại thời điểm trả lời; không đồng nhất với memory.
Ownership	Quan hệ business xác định user nào được truy cập thread.

Tài liệu tham khảo

- LangChain (2026a). *Add and manage memory*. URL: <https://docs.langchain.com/oss/python/langgraph/add-memory> (truy cập ngày 01/08/2026).
- (2026b). *Install LangGraph*. URL: <https://docs.langchain.com/oss/python/langgraph/install> (truy cập ngày 01/08/2026).
- (2026c). *LangGraph overview*. URL: <https://docs.langchain.com/oss/python/langgraph/overview> (truy cập ngày 01/08/2026).
- (2026d). *LangGraph persistence*. URL: <https://docs.langchain.com/oss/python/langgraph/persistence> (truy cập ngày 01/08/2026).
- (2026e). *Long-term memory*. URL: <https://docs.langchain.com/oss/python/langchain/long-term-memory> (truy cập ngày 01/08/2026).
- (2026f). *Short-term memory*. URL: <https://docs.langchain.com/oss/python/langchain/short-term-memory> (truy cập ngày 01/08/2026).